

离散数学编程说明书

版本： V1.1

2026-8-1

第一节 命题逻辑及谓词逻辑

01、离散数学中的数理逻辑是训练逻辑思维能力的重要基础之一，现在要求从终端输入逻辑公式，以真值表形式输出该公式的真值？（由于 ASCII 字符集不支持联结词符号，用"! "代表否定联结词" $\neg$ "、用"&"代表合取联结词" $\wedge$ "、用"|"代表析取联结词" $\vee$ "、用"->"代表条件联结词" $\rightarrow$ "，用"<->"代表双条件联结词" $\leftrightarrow$ "或者用 $((P \rightarrow Q) \wedge (Q \rightarrow P))$ 代替，后续题目依然采用该符号）

如：公式 $(P \rightarrow Q) \wedge R$

1) 输入为 $(P \rightarrow Q) | R$

2) 输出

P	Q	R		$(P \rightarrow Q) R$
F	F	F		F
F	F	T		T
F	T	F		F
F	T	T		T
T	F	F		F
T	F	T		F
T	T	F		F
T	T	T		T

提示：无论是 python 还是 C/C++ 均有 Not、And、or 运算，且 $\rightarrow$ 运算可以转换为 $\neg P \vee Q$ ， $\leftrightarrow$ 运算表依据其运算规律 P、Q 真值相同为真值、否则为假。

算法思维：

1) 形式化与抽象建模：把逻辑公式理解为由变元、运算符和括号组成的字符串，并将其内部结构抽象为表达式树或后缀表达式。这训练了将数学符号体系映射为计算机可处理的数据结构的能力；

2) 词法与语法分析：从终端输入字符串，需要识别命题变元（如 P, Q, R）、逻辑常量（T/F）以及联结词（ $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$ 或对应的符号）。处理运算符优先级（如 $\neg > \wedge > \vee > \rightarrow > \leftrightarrow$ ）和括号，本质上是简化版编译前端，锻炼了有限状态机、递归下降等解析思维。

3) 中间表示与求值算法：将中缀公式转换为后缀表达式或抽象语法树后，对一组具体赋值进行求值。这涉及栈的运用（后缀求值）或树的递归遍历（后序遍历计算），直接训练了栈与递归这两种最基本的算法范式。

4) 组合枚举与真值表生成：自动提取公式中所有不重复的命题变元，并生成它们的全部真值指派（n 个变元对应 2^n 行）。这本质上是一个二进制枚举问题，可以通过多重循环、递归回溯或位运算来实现，训练了穷举所有可能性的系统化思维。

5) 数据结构的选择与组织：如何存储变量列表、如何逐行保存真值表结果、如何对齐输出表格等，都需要合理选用数组、列表、字典（映射）等结构，强化学生数据组织与格式化输出的能力。

6) 模块化与分治策略：将任务拆分为“输入读取 $\rightarrow$ 词法分析 $\rightarrow$ 语法分析/构建中间表示 $\rightarrow$ 提取变元 $\rightarrow$ 生成赋值组合 $\rightarrow$ 逐行求值 $\rightarrow$ 打印表格”等独立模块，体现了分治与接口设计的工程思维。

总结：改题目涉及到编译原理、数据结构、组合数学和计算复杂性的基础思想，是一个非常经典的算法思维综合训练题。

示例代码：详见 01.Logical/01.TruthValue.py

02、求任意命题公式的主析取范式和主合取范式？如公式 $(P \rightarrow Q) \leftrightarrow R$

1) 输入 $(P \rightarrow Q) \leftrightarrow R$

2) 输出

P	Q	R	$(P \rightarrow Q) \leftrightarrow R$
---	---	---	---------------------------------------

F	F	F	F
---	---	---	---

F	F	T	T
---	---	---	---

F	T	F	F
---	---	---	---

F	T	T	T
---	---	---	---

T	F	F	T
---	---	---	---

T	F	T	F
---	---	---	---

T	T	F	F
---	---	---	---

T	T	T	T
---	---	---	---

PDNF is :m1,m3,m4,m7

P	Q	R	$(P \rightarrow Q) \leftrightarrow R$
---	---	---	---------------------------------------

F	F	F	F
---	---	---	---

F	F	T	T
---	---	---	---

F	T	F	F
---	---	---	---

F	T	T	T
---	---	---	---

T	F	F	T
---	---	---	---

T	F	T	F
---	---	---	---

T	T	F	F
---	---	---	---

T	T	T	T
---	---	---	---

PCNF is:M0,M2,M5,M6

提示：无论是 python 还是 C/C++ 均有 Not、And、or 运算，且 $\rightarrow$ 运算可以转换为 $\neg P \vee Q$ ， $\leftrightarrow$ 运算表依据其运算规律 P、Q 真值相同为真值、否则为假。

算法思维：

1) 真值表的系统化生成——穷举与二进制编码：将 n 个变元的所有赋值对应为 0 到 $2^n - 1$ 二进制数，训练用整数位运算控制循环和无遗漏、无重复地遍历状态空间的穷举思维。

2) 逻辑公式求值——表达式解析与递归计算：若输入是任意字符串公式，需要先进行词法/语法分析，构造抽象语法树，然后递归后序遍历求值。这训练了编译前端基础和栈与递归在计算中的运用。

3) 范式构造中的编码映射——大、小项的系统翻译：将一组赋值（0/1 序列）自动转换为极小项（合取式）或极大项（析取式），本质上是按位映射：对小项，1 $\rightarrow$ 原变元，0 $\rightarrow$ 否定；对极大项则相反。这强化了模式匹配与条件分支的机械化规则应用。

4) 规范形式的唯一性思想——标准化与等价判断：主析取和主合取范式是唯一的，这提供了一种判断两个公式是否等价的算法：分别求它们的主范式再比较字符串或小项集合。训练了“转换为规范形”这一在数学和计算机科学中广泛使用的策略。

5) 时间复杂度与指数爆炸的直观感知：真值表行数随变元数量指数增长，求主范式在最坏情况下必须处理 2^n 行数据。学生自然会体会到组合爆炸的现实约束，并建立算法复杂度敏感度，为后续学习 NP 问题提供感性基础。

6) 数据结构的聚合与集合运算视角：主析取范式可视为成真赋值对应小项序号的集合，主合取范式则是成假赋值对应大项序号的集合。两者互补于 0 到 $2^n - 1$ 的全集。这种视角将范式计算转化为集合提取与补集运算，训练了不同数学表示之间的灵活转换。

7) 正确性验证的闭环思维：可由真值表直接写出范式，也可反过来从范式通过逻辑演算还原真值表，或检验“主析取范式与主合取范式互为否定的对偶”等性质。这种双向验证与构造测试预言的习惯，是严谨算法设计的重要环节。

8) 模块化与分治策略：将任务划分为：输入解析、变元提取、真值表生成、逐行求值、极小/极大项构造、输出格式化等独立模块，每个模块可单独测试和替换，训练工程化分解复杂问题的能力。

总结：该题目是一个完整的符号计算系统的微缩设计过程，涵盖了逻辑、组合、编译、算法效率和软件工程等多个维度的思维训练。

示例代码：详见 01.Logical/02.PDNF&PCNF.py

03、接上述真值表判断形式，现在要求输入若干命题公式（公式之间用“,”隔开），判断最后一个公式是否为若干输入前提的有效结论。如公式 $(P \rightarrow Q), (Q \rightarrow R), \neg R, \neg P$

1) 输入为 $(P \rightarrow Q), (Q \rightarrow R), \neg R, \neg P$

2) 输出为

P	Q	R	$(P \rightarrow Q)$	$(Q \rightarrow R)$	$\neg R$	$(P \rightarrow Q), (Q \rightarrow R), \neg R, \neg P$
F	F	F	T	T	T	T
F	F	T	T	T	F	T
F	T	F	T	F	T	T
F	T	T	T	T	F	T
T	F	F	F	T	T	T
T	F	T	F	T	F	T
T	T	F	T	F	T	T
T	T	T	T	T	F	T

此时如果最后一列的真值全部为“T”，则证明结论正确。

提示：该问题的求解思路类似上个问题中求解，输入命题公式构成了蕴含式字符串的判断其真值表是否完为真值，否则无法推导出有效结论。

算法思维：

1) 逻辑问题向计算模型的转化：将“有效推理”这一元逻辑概念翻译为可计算的约束条件，前提集合

$\{A_1, A_2, \dots, A_k\}$ ，结论 B，推理有效当且仅当 $(A_1 \wedge A_2 \wedge \dots \wedge A_k) \rightarrow B$ 是永真式。这训练了将符号逻辑问题转化为布尔表达式求值问题的抽象能力。

2) 多公式输入的解析与分离：输入是逗号分隔的多个公式，需要先进行词法及语法分析，识别每个独立的公式字符串，可能还涉及去除空格、检查括号匹配等。这相当于一个微型分词器，强化了字符串分割、异常处理与模块化解析的思维。

3) 变元集合的自动提取与归一化：从所有前提公式和结论公式中提取全部命题变元，去重并排序。这实质上是在构建真值表的“属性全集”，任何遗漏都会导致真值表缺失行，训练了跨公式的信息聚合和集合运算。

4) 组合状态空间的穷举生成：体现了二进制编码控制多重循环或位运算遍历的枚举策略，并让学生直观感受指数规模的增长。

5) 公式求值的复用与缓存：算法可预先解析并构建每一条公式的语法树或后缀表达式，然后对每一行赋值仅作一次求值。这训练了避免重复解析的预计算思想和共享中间表示的优化意识。

6) 有效性判定的逻辑：寻找反例。算法核心是在遍历真值表时检查：如果某一行的所有前提都为真且结论为假，立即输出“无效”并出示该反例赋值；若遍历完所有行均未发现这样的反例，则输出“有效结论”。

7) 真值结果的逻辑解释与输出设计：不仅要给出“有效/无效”的结论，还应能输出详细的真值表，并高亮显示“反例行”（如果有）。这要求设计清晰的数据结构（如保存每行的前提真值列表、结论真值）和可读的输出格式，训练了用户交互与结果可解释性的工程习惯。

8) 边界条件与鲁棒性检验：可能遇到：前提为空（此时结论必须为永真式才是有效结论）、公式包含非法字符、变元超过合理数量（用户可能输入几十个变元导致组合爆炸，需给出提示）等。这增强了防御性编程和异常处理的思维。

总结：该命题将逻辑推理的机械判定这一深刻的元逻辑问题，拆解为解析、枚举、求值、搜索反例、优化早停等一系列组合算法步骤，是离散数学与程序设计的典型融合。

示例代码：详见 01.Logical/03.Implication.py

04、（谓词逻辑 $\exists$ 、 $\forall$ 的使用）存在与全称量词组合使用的实例：小镇上有 N 个人，可能有一个人是法官，如果有 $N-1$ 个人信任一个人的，且这个人不信任任何其他入，那么这个被信任的人就是法官，否则小镇上没有法官。

提示：设置命题函数 $\text{Trust}(x,y)$ 表示 x 信任 y ，命题翻译为： $\forall x \exists y (\text{Trust}(x,y) \wedge \neg \text{Trust}(y,x)) \wedge (x \neq y)$

算法思维：将谓词转换为类似于图的“入度”和“出度”概念。法官的出度为 0，入度为 $N-1$ 。

1) 谓词逻辑到计算约束的转化：自然语言中的“所有其他人信任他”与“他不信任任何人”本质是入度和出度的全局约束。这要求学会将一阶逻辑的全称和存在量词，转化为图论中度数的计数条件，完成从逻辑描述到计算模型的关键映射。

2) 数据结构选择与信息压缩：信任关系本质是一组有向边。如果直接存为邻接矩阵，空间为 $O(n^2)$ ，示例代码中的优化方式是使用两个一维数组 `indegree` 和 `outdegree`，将全部信息压缩到两个整数数组中，显著降低空间与时间开销。

3) “一次扫描”式的聚合计算：无需对每个人单独遍历所有边来验证逻辑条件。只需遍历一次所有信任边：对每条边 (a,b) 执行 `outdegree[a]++`，`indegree[b]++` 即可，然后再遍历一次，寻找 `indegree[i]=N-1` 且 `outdegree[i]=0` 的人。是聚合-筛选模式代替了逐人嵌套循环，是典型的**复杂度降维**思维。

4) 唯一性的隐含推理与早停：示例代码中一旦找到满足条件的候选者，即可立即返回或停止搜索，无需担心遗漏另一个合法法官。这训练了利用数学性质简化算法逻辑和早停优化的能力。

5) 时间复杂度与可扩展性分析。

6) 输入解析与逻辑分离：如果输入是以某种格式（如列表、矩阵文本）给出信任关系，需要先解析并提取出 N 和边集合。将解析模块与核心逻辑模块分离，训练了模块化设计和输

入输出解耦的工程习惯。

总结：这是经典问题，把存在/全称量词的组合、图论度数模型与线性扫描算法自然地糅合在一起，是训练从逻辑规范到高效算法这一核心计算思维路径的典型范例。

示例代码：详见 01.Logical/04.Quantifier_compment.py

05、（谓词逻辑和前束范式）多重量词嵌套使用的实例：非空集合 A, B, C 上构造的二元关系 $\langle x, y \rangle \in A \times B$ ，且 $\langle y, z \rangle \in B \times C$ ，判断是否所有的 $\langle x, z \rangle \in A \times C$ ？

提示：将命题的形式转换为前束范式为 $\forall x \forall y \forall z (R(x, y) \wedge R(y, z)) \rightarrow R(x, z)$ ，构造 A, B, C 三个非空集合，确定 $R1$ 是 A 集合到 B 集合的关系， $R2$ 是 B 集合到 C 集合的关系，且非空。采用一种三重循环的方式遍历 $R1$ 和 $R2$ 关系： $R1$ 的值域值与 $R2$ 前域值相同，构造新关系 R 是否 $\text{in } A \times C$ 。

算法思维：

- 1) 本质上还是一个谓词公式词法分析和计算的过程，于上述案例相似。
- 2) 从“存在量词”到“搜索过程”的转化，对应图的两跳路径查询，或者关系数据库的自然连接。
- 3) 量词顺序与计算复杂度的关系：通过三重循环遍历，如果发现 $R(x, y)$ 为真且 $R(y, z)$ 为真，但 $R(x, z)$ 为假，则打破全称量词的成立条件（利用短路求值提前返回）。
- 4) 从逻辑蕴含到程序断言与测试：模型检查和测试用例自动生成的思维：给定集合和关系，如何高效地找到一个违反全称命题的赋值。

总结：以该实例为引，设计层层递进的问题，讨论大数据量下的优化策略，从而将数理逻辑与计算思维深度融合。

示例代码：详见 001.Logical/5.Multi_Quantifier.py

***06、（谓词逻辑）谓词逻辑推理，**现在要求从终端输入若干谓词公式，中间用逗号隔开，证明最后一个公式是前 $n-1$ 个命题公式的有效结论。（操作符规则如前述，另外约束为了避免歧义，用 A, E 分别表示全称量词 $\forall$ 、存在量词 $\exists$ ）。

提示：谓词逻辑与命题逻辑完全不同，谓词逻辑的有效性判定是“半可判定的”(semi-decidable)，本质上是不可判定问题。其复杂度为 $O((2^n)^2)$ ，难点在于再限定的集合内全称量词 $\forall$ 为有限的合取，存在量词 $\exists$ 为有限析取式，本质上退化为命题逻辑形式，计算量非常大。因此需要借助于 AST。

算法思维：

- 1) 形式语言的解析与抽象语法树构建：分层，构造 AST，即复杂问题的拆解与分层。按照谓词逻辑 $\rightarrow$ 有限域展开 $\rightarrow$ 命题逻辑 $\rightarrow$ 真值表的逻辑拆解和分层路径，例如： $\forall x P(x)$ 在论域 $\{a, b, c\}$ 上等价于 $P(a) \wedge P(b) \wedge P(c)$ ， $\exists x P(x)$ 在论域 $\{a, b, c\}$ 上等价于 $P(a) \vee P(b) \vee P(c)$ ，就变成了三个独立的命题公式；
- 2) 多重量词嵌套与变量作用域管理：作用域分析、变量重命名、约束变量与自由变量的区分。谓词推理的核心难点在于量词的嵌套。
- 3) 作用域与变量替换，处理 $\forall x \exists y P(x, y)$ 时，展开外层 x 会得到 $\exists y P(a, y), \exists y P(b, y), \dots$ 因此，要递归地把 AST 中所有绑定变量替换为常量，这就是 λ 演算中的替换操作。
- 4) AST+ 环境 (Env) 双结构，AST 表达公式的静态结构，Env 保存变量的动态绑定。这是编译原理中解释器的经典范式。

5) 边界控制于组合爆炸，控制逻辑指数级上升，因此测试时选择小的论域空间（3~5）个为宜；

6) 若找到某个赋值使前提为真、结论为假，就构造了一个模型的反例，则证明推理无效。

总结：该实例是一个浓缩的计算逻辑项目，它从终端字符开始，最终输出“yes/no”及证明路径，完整覆盖了形式语言→符号处理→搜索推理→算法优化这条计算思维主线。通过动手实现，不仅能深入掌握谓词逻辑的语义与证明论，更能在算法设计、复杂状态空间管理和启发式策略上获得扎实训练，为理解现代定理证明器、程序验证、甚至逻辑编程语言打下基础。

示例代码：详见 [01.Logical/06.Predicate_Implicit.py](#)

第二节 图论基础

07、通过键盘输入顶点数 n 和边数 m ，分别随机生成有向图和无向图的边及整数权重，然后绘制二维图形并输出对应的邻接矩阵。（代码包含输入验证）确保边数不超过最大可能值。

提示：

1) 终端输入 n 和 m 的值，分别是顶点的数目和边的数目，考虑到同时生成无向图和有向图，边的数目 m 不超过 $\frac{n(n-1)}{2}$ ，这 m 条赋予 1~10 的随机整数权重。

2) 输出格式是以邻接矩阵以整数形式打印，边权重标注在图中。

算法思维：

1) 组合约束与边界条件验证：输入合法性检查、极值分析，将数学公式转化为防御性代码的思维。若允许自环或多重边，约束会变化，要求设计灵活的上限计算逻辑。

2) 不重复边的随机生成策略：随机生成 m 条不同的边，朴素方法可能产生重复，需要借助集合去重。

3) 邻接矩阵的构建与表示：稠密矩阵操作、对称性处理、特殊值约定。

4) 异常捕获、输入循环、用户友好提示：需校验顶点数、边数是否为正整数，权重是否为整数，以及 m 是否超限。若输入无效，应循环提示而非崩溃。

5) 从问题到程序的完整流程：思维训练：模块化分解、测试驱动；任务可拆分为：输入模块、验证模块、边生成模块、矩阵构建模块、可视化模块；每个模块可独立测试，例如用固定种子验证生成结果，检查无向图矩阵对称性等。整体上锻炼“分析需求→设计模块→编码→测试”的工程化思维。

总结：该实例以随机图生成成为载体，串联了组合约束验证、高效去重采样、数据结构抽象、几何布局计算、异常处理等多个基础算法与工程实践要点。通过动手实现，可强化对图论概念的程序表达，并深刻体会“正确性”与“效率”之间的权衡，是从理论走向实际开发的理想训练项目。

示例代码：详见 02.Graphics/01.CreateGraphgics.py

08、有向图的强连通判断，生成一个有向图并构建一个图矩阵，采用判断连通图的基本迭代式方法判断该图是否为一个强连通图。

提示：上题中生成了一个有向图，在改图中所有的权重值设置为 1，表示两个顶点之间是连接的，根据迭代式算法或者 warshall 算法，迭代求解最后生成的矩阵是否全部元素（除了主对角线）全部是 1？

算法思维：

1) 图结构的矩阵代数化——从图到矩阵的映射思维：将图用邻接矩阵或可达性矩阵表示，边的存在变成布尔值 0/1 或者权重值。

2) 迭代式求解有向图的强连通结构采用的迭代式算法是采用矩阵相乘（不是点乘）的方式，首先对初始化的矩阵 A 进行一次计算 $A \times A$ ，得到 A^2 ，依次得到 $A^3 \dots A^n$ （其中 n 是顶点的数值，也是矩阵行列值）；

3) 强连通性的矩阵刻画——全局条件检验：执行 A^i 矩阵的布尔相加，得到的最终结果判断：除了主对角线元素为 0，其余元素全部为 1，则是强连通图，否则不是强连通图。

4) 时间复杂度与稠密/稀疏权衡——算法选择意识：迭代式方法的时间复杂度为 $O(n^4)$ ，warshall 算法（第三章介绍关系闭包时会详细分析）的时间复杂度为 $O(n^3)$ ，如果设置顶点数非常大，则会出现计算量暴涨的现象。

5) 迭代过程的中间状态可视化——调试与理解：在迭代中输出每一步的可达矩阵，观察“1”是如何逐渐扩散的，直观理解“传递闭包”的蔓延过程。训练通过可视化中间数据来调试和解释算法的思维，尤其适合教学和排查逻辑错误。

总结：这个实例将图论、矩阵运算、动态规划、位运算优化和测试验证编织成一个完整的算法训练链。它让学生从“调用库函数判断连通性”的浅层思维，深入到“如何用基本迭代一步步构造出全局性质”的计算内核，培养形式化抽象、算法设计、复杂度分析和工程优化四维一体的综合能力。

示例代码： 详见 02.Graphics/02.strongly_connected.py

09、Hierholzer 算法（又称“套圈法”）是一种用于在欧拉图（所有顶点度数均为偶数）中寻找欧拉回路（遍历每条边恰好一次并回到起点）的高效算法。它也可稍作修改用于存在恰好两个奇度顶点的半欧拉图，寻找欧拉路径。

提示：从图中的任意起点出发，沿着未访问的边走，直到回到起点（形成一个简单环）。若存在边尚未访问，则从当前环上某个仍有未访问边的顶点出发，再走一个新环，并将其插入到原环的对应位置。重复上述过程直至所有边被访问，最终得到一个完整的欧拉回路。

算法思维：

- 1) 复制邻接表，避免破坏原图。选任一顶点作为起点，若图连通且全偶度，任意点均可。
- 2) 模拟 DFS 路径，以及一个列表 `circuit` 存储最终欧拉回路（逆序）。起点做压栈操作，并令 `curr = start`。
- 3) 循环直到栈为空：
 - (1) 若当前顶点 `curr` 还有邻接边（`graph[curr]` 非空）：
 - 将 `curr` 压入栈（表示路径回溯点）。
 - 从邻接表中取出一个邻居 `nxt`，并删除该边（双向删除）。
 - 令 `curr = nxt`，继续深入。
 - (2) 否则（无可用边）：
 - 将 `curr` 添加到 `circuit` 尾部（此时该顶点没有未访问边，可以闭合当前环）。
 - 从栈中弹出上一个顶点作为新的 `curr`（回溯）。
- 4) 每次深度优先搜索都会找到一个闭合环（因为图是欧拉的，走到死路时一定回到起点）。利用栈，当一条路径终结时将其加入结果，然后回溯到最近仍有未访问边的顶点，继续探索新环。最终所有边被访问，且所有环通过回溯点自然连接成完整回路。
- 5) 复杂度分析：时间复杂度：每条边被访问两次（正向和反向删除），故为 $O(E)$ ；空间复杂度：存储图 $O(V + E)$ ，栈和路径 $O(E)$ 。

算法的思考：采用逆序是因为在无路可走时才将顶点加入 `circuit`，类似于后序遍历，最终的序列是反转后才符合访问顺序。

示例代码： 详见 02.Graphics/03.Hierholzer.py

10、无向图判断是否为欧拉图，是否为哈密顿图，并且以图示结构输出是否存在欧拉回路和哈密顿回路。

提示：欧拉图本质是每个边只经过一次，并且起点和终点相同；哈密顿图的本质是每个点经过且只经过一次，起点和终点相同。

1) 理论上分析只需要判断所有顶点的度数（无向图）为偶数即可，但有可能会漏掉某些特殊的欧拉图（定理 7.5.2 所述）。

2) 哈密顿图中加入路径的节点与前一个加入路径的顶点一定是连通的；加入路径的最后一个顶点和第一个顶点一定是连通的；但是路径上不能出现重复的顶点。

算法思维：

1) 图论定理与算法化转译：将数学判定条件（欧拉定理）直接翻译成程序检查：连通性+度数全偶。训练将数学性质转化为可执行的判断代码的思维。

2) 复杂度与问题难度的直觉：欧拉回路 $O(n + m)$ ，哈密顿回路 NP 完全。同一个图结构，两个问题难度差别，强化对 P 和 NP 的直观理解，培养选择合适算法的意识。

3) 回溯与剪枝——状态空间搜索：哈密顿回路必须用回溯。状态空间树的构建（路径序列）；剪枝策略设计（度数、连通性、死端检测）以缩小搜索空间；递归与栈的管理，回溯时撤销操作。

4) 边标记与路径回推——欧拉回路的“拆环”思想：Hierholzer 算法通过先找环再拼接环，本质是分治法。训练：如何处理“边走边删”而不影响未走过的边（可用边列表迭代器或邻接矩阵）；后进先出的栈拼接回路，理解深度优先搜索的括号结构；

5) 两种回路的可视化校验：输出回路后，还需程序自身验证回路正确性（边不重复、顶点全覆盖、闭环）。训练自我测试和断言的使用。

6) 算法选择与实际问题规模 n 稍大的哈密顿图，回溯可能指数爆炸。训练引入启发式/近似算法的思维（如使用动态规划 Held-Karp 对 n 不大时精确求解），并讨论何时必须退而求其次。

7) 数据结构对算法效率的影响：欧拉删边操作：用邻接表 + 索引指针（或双向链表）可高效删除。

8) 从数学定义到交互式图形：将抽象回路以视觉方式呈现，需要把顶点坐标映射、边绘制、标签显示等转化为代码，锻炼计算几何与可视化结合的综合能力。

总结：该实例通过对比欧拉回路（多项式可解）与哈密顿回路（NP 完全），在一个程序中展示了图论、算法设计、状态空间搜索、剪枝优化、可视化输出等多层知识，是训练计算思维的经典范例之一。

示例代码：详见 02.Graphics/04.EULA_Hamiltonian.py

11、图论在分析最小生成树的算法采用的是 kruskal 算法，提供排序后的最短路径（可以调用 sort 方法），寻找最小的路径长度，判断是否有回路，直至构造一条 $n-1$ 个分支的图，且该值是所有生成树中权值最小的。

提示：Kruskal 算法用于求解无向连通图的最小生成树（MST, Minimum Spanning Tree）采用了贪心策略：1) 将所有边按权重从小到大排序；2) 依次尝试将每条边加入生成树，如果加入后不会形成环，就保留这条边；否则丢弃；3) 当加入的边数等于顶点数-1 时，最小生成树构建完成。

算法思维：

1) 贪心策略与最优子结构：理解 Kruskal 算法为何每次选择当前最小权边是“安全”的。证明依据是割性质：任取一个割，连接该割的最小权边必然属于某个最小生成树。

Kruskal 算法每选一条边，实际上都在某个割上做了最优选择。

2) 排序预处理与“离线”思想：初始化是指每个顶点自成一个集合（并查集初始化），后排序，将图中的所有边按权值升序排列，（可以根据冒泡排序的演化形式快速排序）

3) 并查集判环——不相交集的高效实现：判环的本质是检测当前边的两端是否已连通。用并查集（Union-Find）可在近乎 $O(1)$ 时间内完成合并与查找。路径压缩与按秩合并两个优化将理论复杂度降至反阿克曼函数级别，训练数据结构优化对算法效率的质变影响。深入理解并查集如何将图的动态连通性问题转化为树形结构操作。

4) 边权相同与多解问题：当存在多条等权边时，不同选择顺序可能导致不同的最小生成树（但总权和相同）。引出 MST 唯一性的讨论，训练算法对不确定性的敏感性分析，以及如何处理平局策略。

5) 实际应用中的映射思维：最小生成树可用于网络布线、聚类分析、近似求解旅行商问题等。将真实场景抽象为“无向有权图求 MST”的过程，训练问题建模与算法实现的能力。

总结：Kruskal 算法通过排序+贪心+并查集这一组合，完美诠释了数学性质（割）、数据结构（并查集）与算法设计（排序预处理）三者协同配合的计算思维。深入掌握它，能有效训练贪心论证、数据结构优化、复杂度分析和循环不变式等关键能力。

示例代码：详见 02.Graphics/04.Kruskal.py（在代码中给出了测试集合）

补充训练示例 1：Kruskal 算法及画出最小生成树

示例代码：详见 02.Graphics/ 05.Kruskal_Algrithm.py

示例代码：详见 02.Graphics/06.Kruskal2MST.py

补充训练示例 2：prim 算法计算最小生成树

示例代码：详见 02.Graphics/ 07.Prim_Algrithm.py

第三节 集合与关系

12、集合的并交差补运算，要求非空集合 A 和 B，都是简单{a-z}内字符元素组成，随机生成 A、B 两个集合，个数各自不超过 5 个元素，设 $E = \{a \sim z\}$ 为全集，写代码进行并交差补运算，并展示最终的运算结果。

提示：简单集合（不包含子集）运算，包括并交差、补运算，可以借助于 python 中 string 包的函数和方法实现代码，但是所有算法要求用代码形式实现，不允许调用封装的方法。

算法思维：

1) 集合的抽象数据类型与底层实现：使用 set 类型（哈希表）存储元素，实现自动去重和快速成员判断 $O(1)$ 。训练选择合适数据结构的思维：可用布尔数组（[26]）表示字母集合，权衡时间与空间。

2) 随机生成满足约束的子集：需从 26 个字母中随机抽取非空且元素数不超过 5 的子集。

3) 集合运算的运算符/方法映射：并 |、交 &、差 -、补（通过差实现），将数学符号直接转换为代码。训练形式化语言与编程语言的无缝转译，并理解运算符重载背后的集合代数性质。

4) 输出结果的可读性与格式化展示。

5) 边界条件与异常情况处理：非空输入生成后检查集合是否为空，否则重新生成或调整参数。全面的测试思维，覆盖空集、满集、相等集等临界情形。

6) 扩展思维：引出集合运算的并行化：若数据巨大，差集如何分布计算？训练从玩具代码到大规模系统的扩展意识。

总结：强调将离散数学概念精确实现为代码，并通过输出来直观感受集合运算的结果，是训练“数学形式化 $\leftrightarrow$ 程序实现”双向转换能力的案例之一。

示例代码：详见 03.SET/01.SetCalulate.py

#测试用例，随机函数生成

13、集合关系的判断，给定两个不同的非空集合，判断两个集合之间是包含、相等或者交集为空？约定两个不同的非空集合 A 和 B，字符集在{a-z}或者{0-9}取值，不含特殊字符，终端输入两个不同集合元素，以"}"为结束标志，在终端输出两个集合关系判断的结果。

提示：该题目借助于顺序查找操作判断两个集合之间的关系，按照简单集合的构造形式（不包含子集、空集和重复元素），通过两个列表查找判断。

算法思维：

1) 顺序查找是数据结构和算法中的基础算法，要求是遍历整个表列，进行元素的查找操作，python 中提供了 in 操作，虽然简化了循环操作，但建议写代码时自己完成 in 操作的代码。

2) 有限状态机的设计思维：读取状态（等待集合 A、读取集合 A、遇到 A 的结束符、等待集合 B、读取集合 B、结束）。

3) 集合数据结构的选择与去重：集合元素天然不重复，输入可能包含重复字符，需要自动去重。

4) 集合关系判定的算法设计：包含判断 $A \subseteq B$ 等价于遍历 A 中每个元素检查是否在 B

中，一个不在即返回假。相等判断：先判断大小是否相同，再调用包含判断（或直接比较集合相等）。交集为空判断：遍历较小集合，任一元素出现在另一集合即非空，可提前终止。

5) 决策逻辑与关系互斥性处理：关系可能重叠，需定义优先级：相等优先输出，其次严格包含，再交集为空，否则输出“相交但不包含”。设计清晰的分支结构或策略模式，训练将逻辑规则精确翻译为控制流的能力。

6) 测试驱动的思维与边界覆盖：典型用例：完全相等、真子集、不相交、部分相交、重复字符、单个元素集合、字母数字混合等。可设计断言自动化测试，验证各种场景，培养“用代码验证代码”的习惯。

7) 算法效率与常数优化。训练短路求值和基于大小的优化（选择遍历小集合以降低复杂度）。

总结：该实例从终端字符解析到关系逻辑输出，串联了输入处理、数据结构选择、集合算法、逻辑分支、鲁棒性设计等多重基础技能。它强调将离散数学中的集合论概念直接转化为可运行的、健壮的程序，是训练“问题→抽象→编码→测试”全流程思维的案例之一。算法设计时要考虑算法的时间复杂度，能否把算法降到 $O(mn)$ 范围内？

示例代码：详见 03.SET/02.Set_and_Ralationship.py

14、设 $R1$ 和 $R2$ 都是非空集合 $A \times B$ 上的两个非空关系，且 A 、 B 两个集合在 $\{a-z\}$ 或者 $\{0-9\}$ 取得字符集，要求键盘输入 $R1$ 和 $R2$ 两个关系完成两个关系 $R1$ 和 $R2$ 是否相等的判断？并输出最终判断结果。

提示：为了区别 A 和 B 两个不同集合，分别使用随机数产生 $\{a-z\}$ 和 $\{0-9\}$ 置于 A 、 B 集合，根据 $|A|$ 和 $|B|$ 随机生成 $R1$ 和 $R2$ 关系。该问题本质上还是一个查找算法的实际应用，鉴于 $R1$ 和 $R2$ 在定义中不存在序列问题，依然采用一种顺序查找的方式。

算法思维：

1) 问题抽象与建模能力：将“关系”这一数学概念映射为有序对的集合，把“判断两个关系是否相等”抽象为“判断两个集合是否包含完全相同的元素”。这是从业务描述到计算模型的转化。

2) 数据结构的选择与运用：需要选用合适的数据结构来表示关系——通常使用集合（如 Python 的 `set`）存储元组，利用集合元素无序、唯一的特性，直接支持相等性判断；或者用列表排序后比较。选择的过程训练了对哈希表、无序集合特性的理解。

3) 输入解析与格式化处理：键盘输入可能是类似 $\{(a,1),(b,2)\}$ 的字符串，要求设计解析逻辑，提取有序对并处理分隔符、空格、括号等。这一步强化了字符串分割、模式识别和类型转换的思维。

4) 无序集合相等性算法：判断两个集合相等的核心算法：可以先比较元素个数，若不等则直接判定不等；若相等再逐个检查一个集合的元素是否全部属于另一个集合（子集包含）。训练“先快速否定，再细粒度验证”的优化意识。

5) 时间复杂度与效率：使用哈希集合可以将单次查找做到平均 $O(1)$ ，整体 $O(n)$ ；如果采用排序后逐对比较，复杂度为 $O(n \log n)$ 。通过这一选择，会自然关注到不同实现方式的效率差异。

6) 边界条件处理与验证：虽然命题限定为非空关系，但实际输入可能包含多余空格、重复有序对、不符合字符集规定的元素等。需要考虑去重、输入合法性校验，训练防御性编程

和异常处理的思维。如果两个关系内容相同但顺序不同、大小写差异、包含重复定义、元素部分重叠等，全面验证程序正确性，培养构造测试场景的习惯。

该命题是一个小型的综合性练习，串起了“数学抽象→数据表示→输入解析→算法设计→复杂度分析→代码健壮性”这一完整的思维链条，非常适合用来训练计算思维和基础算法设计能力。

示例代码：详见 03.SET/03.Relationship_Equ.py

15、R 是 A 集合的二元关系，求 R 的自反、对称和可传递闭包。

提示：首先输入 A 集合元素的个数（即构造的关系矩阵的行列值），随机生成任意关系，然后根据相关的定理，生成自反对称和可传递闭包（迭代式和 warshall 方法均可）

算法思维：

- 1) 理解计算机伪随机数的生成，分散到对应矩阵中的相应位置（位置也是随机）。
 - 2) 闭包定义的构造性转译：自反闭包添加所有缺失的对角线元素，对称闭包添加所有反向对，传递闭包反复补充传递链。训练将数学上的“最小包含关系且满足性质”这一非构造性定义，转化为构造性算法步骤的能力。
 - 3) 关系矩阵化与代数操作：用布尔矩阵表示关系后，自反闭包是对角线置 1，对称闭包是矩阵与其转置的逻辑或。传递闭包则对应矩阵的传递闭包运算（布尔幂并集）。
 - 4) Warshall 算法的动态规划思想。
 - 5) 正确性验证与最小性检验：编程后需验证结果满足对应的性质（自反、对称、传递），并确保是“最小”的——去掉任何一对关系就不再满足该性质。
 - 6) 扩展算法有效思维：矩阵稀疏性与数据结构选择：若关系非常稀疏（如 A 很大但 R 很少），邻接矩阵浪费空间，邻接表配合 BFS/DFS 求传递闭包可能更优。引出位向量并行（Python 的 int 作为位集）加速布尔矩阵运算。
- 总结：该实例加强学生深刻理解离散数学结构如何经由算法思想（动态规划、表示转换）转变为高效程序，是连接数学与计算思维的桥梁。

示例代码：详见 03.SET/04.Closure_calculate.py

16、迭代式方法和 warshall 方法是计算集合 A 上的任意二元关系的传递闭包求解常用方法，但是迭代式求解方式计算时间复杂度为，warshall 算法复杂度为。给一个（ $100 < l < 1000$ ）的数值，生成一个 $l \times l$ 矩阵，分别计算其各种耗费计算时间，通过比对计算时间辅助理解时间复杂度的概念。

提示：终端输入一个大于 100 且小于 1000（不宜太大，否则会造成程序运行崩溃现象），调用随机函数生成一个的 $l \times l$ 矩阵，根据另外一个输入的随机数 m，是该矩阵中 1 元素的个数，随机分别到该矩阵中。

算法思维：

- 1) 从理论公式到实验验证的思维闭环：训练“假设→实验→观察→印证”的科学方法，而非仅停留在数学推导层面。
- 2) 不同规模下的增长趋势敏感度：取 $l=100、200、500、1000$ 等不同规模分别实验，绘制时间-规模曲线，可直观感受 多项式时间内的“陡峭”与“平缓”。
- 3) 算法设计优劣的决定性作用：同一个问题（传递闭包），迭代法穷举路径长度，Warshall

算法动态规划逐步解放中间节点，结果相同但效率相差一个数量级。体现了“算法，而非硬件，是效率的根本” 计算思维核心。

4) 不动点迭代的收敛行为观察：迭代法中，每次循环是在更新“长度为 k 以内的路径”，可输出迭代次数 观察收敛速度。

5) 存储结构与位运算的底层优化意识：布尔矩阵的每一行可用一个 `bitset`（位向量）表示，则矩阵乘法中的内层循环可被 64 位或更大宽度的位运算一次性完成，极大降低常数。

6) 问题规模对算法选择的指导意义。根据问题规模预估运行时间、灵活选算法的工程决策模型。

总结：该实例通过一次可控的对比实验，将 时间复杂度分析、算法正确性验证、性能测量、工程优化四个维度串联，是训练计算思维从理论走向实践的绝佳载体。

示例代码： 详见 `03.SET/05.Iterative_Warshall.py`

17、设非空集合 A 上的任意二元关系 R ，从自反、对称和可传递三个性质方面来判断该关系性质，包括等价关系、相容关系、偏序关系判断。

提示：终端输入 A 集合元素的个数 n 和 A 集合上关系 R 的组数 m ，且一定要满足 $m \leq n^2$ ，否则无法形成 A 的集合映射。构造 R 的关系矩阵，根据矩阵在自反、对称上的特征判断是否满足该特性。遍历关系 R 检测是否有 $\langle x, y \rangle \in R$ 并且 $\langle y, z \rangle \in R$ ，是否存在 $\langle x, z \rangle \in R$ ，如果有继续搜索，否则不满足传递特性。

算法思维：

1) 数学定义到程序逻辑的直接映射：将“对任意 x ，均 xRx ”转译为遍历所有元素检查对角线；将“若有 xRy ，则必有 yRx ”转译为对每对元素检查双向存在性；将“若有 xRy 且 yRz ，则必有 xRz ”转译为对每三个元素检查传递链。

2) 短路求解方法：判断对称性时，一旦发现单向边立即返回假；判断传递性时，一旦找到传递违例立即终止。若不自反则绝不可能是等价/偏序关系，从而跳过后续的检查。训练提前剪枝的习惯，减少不必要的计算。

3) 传递性判定的算法选择与复杂度分析。

4) 反例构造与可解释性输出：程序不仅输出“是/否”，还应提供反例：例如“关系不是传递的，因为存在 (a, b) 和 (b, c) ，但没有 (a, c) ”。训练算法调试与结果解释能力，是构建可信系统的重要思维。

5) 边界条件与极端测试的设计：空关系、恒等关系（仅对角线）、全关系、等价关系与偏序关系的典型实例（同余类、整除）均可作为测试用例。训练测试驱动的正确性验证，确保算法在各种极端输入下符合数学定义。

总结：该实例将一阶谓词逻辑、矩阵运算、图论、组合优化巧妙融合在一个编程任务中，通过“定义→判断→分类→反例输出”的完整流程，训练了严密的逻辑形式化转译能力、算法复杂度权衡能力以及工程化测试思维，是离散数学与计算思维结合典型案例之一。

示例代码： 详见 `03.SET/06.Relationship_properties.py`

***18、**最大子序列问题：偏序集上的最长链问题，是指在无序序列中，寻找严格递增的子序列

的最大长度，元素间不一定是连续的，但是其相对顺序不能发生改变。

例如：

输入：[10, 9, 2, 5, 3, 7, 101, 18]

输出：4 （对应 [2, 3, 7, 101] 或 [2, 5, 7, 101]）

提示：最大子序列问题与 Dilworth 定理密切关联，一个偏序集的最长链长度（LIS），等于它能划分成的最少反链数量（最少递减子序列覆盖数）。本质上是最长递增子序列的长度就是最少递减子序列的个数。定理的**对偶**形式即任何有限偏序集中最长链中元素的数目等于最小反链划分中反链的数目。

算法思维：采用动态规划法，时间复杂度 $O(n^2)$ ，思维实现的逻辑：

1) 最优子结构与动态规划设计：初始化建立一个长度与序列相同的数组例如示例代码中的 `dp[]`。由于每个数字自己单独就能构成一个长度为 1 的子序列。设立一个变量为 1 记录全局答案。

2) 贪心+二分查找结合：算法将“最长递增”转化为“维护一个单调结构”，训练问题转化和贪心维护最小值以保留未来扩展潜力的策略思维。

3) 从偏序关系到算法抽象的升华：LIS 本质是在“下标增序”和“值域增序”两个全序约束下求最长链。可推广到 DAG 上的最长路径，即偏序集上的 Dilworth 定理或 Mirsky 定理。训练从具体问题提炼抽象结构的能力，理解同一算法在不同领域的映射，如任务调度、信封嵌套、序列比对等。

4) 多种解法的复杂度和适用性对比：对比不同算法在时间、空间、可扩展性（如能否输出具体序列）上的权衡，训练工程化算法选择能力。

5) 扩展思维之对偶问题与反向思维：求最长递增子序列等价于求序列的反向最长递减子序列，也等于序列长度减去最少删除个数使序列有序。引出 Dilworth 定理：最长链的大小等于最小反链覆盖数。这训练对偶转换思维，学会从不同角度定义问题以简化求解。

6) 算法逻辑思路采用了两层循环控制：

①外层循环变量 `i` 控制当前要比对的元素 `list[i]`;

②内层循环变量 `j` 采用穷举的方法，判断当前位置 `i` 元素大于前面的元素 `j` 满足递增条件的话，`list[i]`接在以 `list[j]`结尾的序列后面；

③更新 `dp[j]` 的值，标记当前最大值；

④内层循环结束后，记录结果，赋值给 `dp` 最大的值

⑤更新全局最大值，把 `dp[i]` 和全局最大值比较，保留大的。直至外层循环介绍，找到最大序列值。

总结：该实例串联了动态规划、贪心策略、二分搜索、偏序抽象、回溯构造与对偶转化等核心算法思想，是从基础算法走向高级问题求解的枢纽性训练实例。

示例代码：详见 03.SET/07.MaxList.py

第四节 函数和代数系统

19、集合 A 上的笛卡尔积子集组成的二元关系 R ，可以构造为函数，要求是构造一个可逆的函数。

提示：集合 A 上的二元关系 R 构造采用随机数构造，保证构造函数的真实性和可信度。基于该关系 R 从是否满足满射和入射两个性质做出判断，即构造为一个双射函数：

- 1) $f: A \rightarrow A$ ，满足任意性和结果唯一性；
- 2) $f^{-1}: A \rightarrow A$ ，同样也满足任意性和结果唯一性；

算法思维：

1) 关系与函数的程序化判别：函数要求每个 x 在 R 中至多有一个 y 对应，可逆还要求每个 y 至多有一个

x 对应。训练学生将数学定义转化为遍历检查和有效性验证的代码，并处理“一对多”冲突时的选择策略。

2) 图论建模：二分图匹配将集合 A 复制为左右两部， R 中的每对 (x,y) 作为左 x 到右 y 的边。构造可逆函数等价于寻找一个完美匹配（覆盖所有顶点的匹配）。训练将抽象代数结构转化为图论模型的映射能力。

3) 双射构造算法与搜索策略：) 或最大流算法在二分图中求解完美匹配，若存在则输出双射函数，否则报告无解。若自行实现回溯搜索，需为每个 x 分配一个 y ，并处理冲突时的撤销（回溯），训练深度优先搜索 + 状态恢复的技巧。

4) 确定性贪心与随机化尝试：当关系稠密时，可先用贪心消去度数为 1 的顶点（确定性匹配），减少搜索空间；也可随机打乱顺序多次尝试避免局部阻塞。训练预处理剪枝和随机化重试在组合构造问题中的应用。

5) 可逆性的验证与逆函数构造：构造完成后，需验证是否为双射：检查定义域、值域是否覆盖整个 A ，且无重复映射。逆函数只需交换所有有序对的顺序。训练函数性质自动化测试及对合操作（求逆）的实现。

6) 复杂度与可行性的边界分析：当 R 是任意关系时，完美匹配不一定存在，程序需能判断并输出“无法构造”。理解完美匹配存在的充要条件（详见 Hall 定理），并将定理条件转化为可检查的算法。训练数学定理算法化及不可行情况的识别能力。

7) 从部分函数到全函数的扩展思维：最初 R 不包含某些定义域元素的值，需要决定是否允许扩展定义域（添加新映射），这触及函数完备性约束。该实例明确要求基于已有的 R 子集构造，强调了约束满足问题（CSP）的基本形态。

总结：该实例将集合论、函数概念、二分图匹配、搜索与回溯、数学定理验证融为一体，训练了从抽象数学对象到具体算法实现的完整链路，强化了“关系→图→匹配→双射”的计算思维转化。

示例代码：详见 04.Function&Group/00.Judge_is_function.py

20、给定一个抽象代数运算，判断该代数系统是否满足为半群、独异点、群的特性？核心点在定义运算和搜索算法。

提示：以 $[+]_8$ 为例，对于任意在 0-7 中的两个元素，先求加法运算，再进行模为 8 的取余运算，构造了一个二元代数运算。然后再根据半群满足可结合特性、独异点含有么元、群中任意元素都有逆元的特性，判断是否为真。

算法思维：

1) 运算表的抽象表示与封闭性验证：使用二维数组（凯莱表）存储运算结果。封闭性检查即遍历整个矩阵，确保每个元素都在集合内。训练离散结构的程序化表示，理解形式化系统如何映射为数据存储。

2) 全称量词的循环转译：“对所有元素成立”直接转译为遍历循环。结合律需要检查所有三元组 (x,y,z) ，即三重循环，形成直观的 $O(n^3)$ 验证算法。训练将数学上的全称量词 $\forall$ 转化为确定性的穷举验证模式。

3) 存在性目标的搜索与剪枝：

单位元搜索：按定义遍历候选 e ，一旦找到使得所有 x 满足 $e * x = x * e = x$ 的元素即停止；若遍历完未找到则不存在。逆元搜索：在确定单位元 e 后，对每个 x 搜索 y 使 $x * y = y * x = e$ ；一旦某个 x 找不到逆元，该代数系统即非群。训练带目标的存在性搜索与短路剪枝策略，避免不必要的计算。

4) 复杂度观察与优化思考：判断结合律的算法复杂度 $O(n^3)$ 是主要瓶颈。如果运算满足交换律可减少检查量吗？是否可以提前预判（如存在幂等元分布）？对于大 n ，可引入随机测试：先随机抽取大量三元组验证结合律，若全部通过再启动完全验证，以防浪费算力。训练概率验证与确定性验证结合的思路，以及算法复杂度对实际可行性影响的直观认知。

5) 反例构造与可解释性输出：当某个性质不成立时，输出具体反例：如结合律违反的三元组，不满足单位元性质的元素，找不到逆元的元素等。

6) 边界条件与不变式思维：空集、单元素集的特殊情形需正确处理（单元素集上定义适当运算可成平凡群）。可验证一些不变式：单位元若存在一定唯一；逆元若存在且运算结合则一定唯一。训练对数学唯一性定理的程序验证，加深对代数系统结构性质的理解。

总结：该实例通过“定义运算→搜索验证→分层判定”的流程，将抽象代数、全称/存在量词的算法转译、搜索剪枝和复杂度权衡融为一体，是训练数学形式化与计算思维双向转化的典型范例。

示例代码：详见 04.Function&Group/01.Algebra_Caucaite.py

21、复数元素性质：定义集合 $s = [1, -1, 1j, -1j]$ ，运算是复数乘法。编写程序判断该代数系统是否为群，并说明它是循环群吗？由哪个元素生成？

提示：Python 支持复数 $1j$ 。但是需要注意浮点误差，建议使用 `round` 处理或直接用整数对表示（这里直接用复数检验）。判定群后，可通过反复乘某元素看能否生成全集来验证循环群。

算法思维：复数的加减乘运算的代码表示： $(a + bj)(c + dj) = (ac - bd) + (bcj + adj)$ 。该算法思路就是一个 2×2 矩阵的乘法运算的表示。

1) 代数结构的程序建模与封闭性验证：用列表或集合存储元素，定义乘法运算为复数乘法（Python 内置 `complex` 类型自动支持）。封闭性：双重循环计算所有乘积，检查结果是否仍在集合中。这里集合小（4 个），可直接硬编码或动态生成乘法表。

2) 群公理的逐条算法化转译：如上案例所述。

3) 循环群性质与生成元的搜索算法：生成过程中元素的阶，理解有限循环群的生成元搜索本质是穷举+幂运算。训练生成序列的迭代构造和集合相等性判断。

4) 特殊代数结构的恒等式快速验证：由于已知该集合是 4 阶循环群，可直接利用性质：单位元为 1，仅需检查每个元素的逆存在（1 逆为自身，-1 逆为自身， i 与 $-i$ 互为逆）。

5) 边界条件与错误处理：若某性质不成立（尽管本例全成立），应输出具体反例（如哪个三元组违反结合律，哪个元素缺逆元），培养反例构造思维。对于任意复数集合的通用判定程序，需考虑浮点误差，可使用整数或分数表示避免精度问题。本例中元素为纯整数或纯虚数，乘法结果精准，可直接等值比较。

6) 抽象代数到程序逻辑的思维扩展：从具体的运算表出发，程序自动判断半群/独异点/群/循环群，展示了有限代数结构的可判定性和自动推理的基本思想。

总结：该实例通过一个微小的四元复数群，将群公理验证、生成元搜索、运算表构建、穷举与剪枝等计算思维融入代码，是抽象代数与算法思维结合经典案例之一。

示例代码：详见 04.Function&Group/02.Complex_is_Group.py

22、定义集合为所有长度 ≤ 5 的小写字母字符串（可以抽样验证），运算 $op(a,b)$ 返回较长的字符串，长度相等时返回字典序较小者。编写代码验证该运算是否不满足结合律？

提示：编写程序时可以考虑调用 python 自带的 string 包，调用 `len()` 函数检查 `s1,s2` 字符串长度，返回长字符串。如果 `s1` 和 `s2` 字符串长度相同，则返回字典序较小的，例如 `'ab' < 'ac'`。

算法思维：本质是考察代数系统上规定的二元运算 Max ，即取最大值的代数系统是否满足结合律构成半群，参照上例中的计算思维模式。

1) 无限/大有限集合的抽样验证思想：由于全集太大无法穷举，需要采用随机抽样策略，训练如何生成具有代表性的测试用例，并理解抽样验证的概率意义。

2) 反例搜索策略：结合律不成立需要找到反例 a, b, c 。可以通过随机生成三元组并检查等式，直到找到反例。这训练了“基于测试的数学发现”方法。

3) 运算定义的直接实现与性质验证：将运算 op 实现为函数，并直接编码结合律的检查条件。训练将数学定义转化为代码并进行逻辑验证的能力。

4) 字典序比较和字符串操作：运算涉及字符串长度、字典序，这涉及语言内置的字符串比较，培养利用标准库进行自定义比较逻辑的思维。

5) 反例分析与最小化：一旦发现反例，尝试化简反例（如短字符串），从而揭示为什么结合律不成立，训练从具体反例抽象出一般模式的能力。

6) 算法复杂度与抽样次数的权衡：抽样次数越多，找到反例的概率越大，但耗时也更长。训练对于概率验证的置信度与资源的权衡。

7) 自动推理的局限性：表明对于无限域，全称验证不可行，只能通过搜索反例来否定命题，这是自动定理证明中常见的思维。

总结：该实例通过“大有限集合→抽样搜索→反例发现→归因分析”这一流程，将概率验证、测试生成、边界分析、反例构造等算法思维巧妙融合，是从经典数学验证迈入现代计算思维的关键一步。

示例代码：详见 04.Function&Group/03.MaxlengthString.py

23、结合书面作业常见的代数系统，自主构造任意代数系统，验证系统是半群、独异点、群、循环群。例如：构造一个半群但不是独异点的例子：正整数集 $S = \{1, 2, 3, \dots\}$ 运算 $op(a, b) = a + b + 1$ 。用代码（取有限子集）验证结合律，并证明无单位元。构造独异点但不是群的例子：自然数集 $N = \{0, 1, 2, \dots\}$ 普通加法。验证有单位元 0，但非零元素无逆元。（选做）找出一个非阿贝尔群的有限表示，例如模 2 下的二阶可逆矩阵乘法。可构造集合并验证非交换性。

提示：自主构造不同类型的代数系统（半群非独异点、独异点非群、非阿贝尔群），并通过编程（取有限子集）验证其性质。

算法思维：

1) 代数结构的“自构造”与模型设计思维：不仅要验证给定系统，还要主动设计满足特定性质（且不满足另一些性质）的运算与集合。这训练从性质目标出发逆向构造模型的能力，是数学建模与算法设计的核心。

2) 无限集到有限子集的近似与局限意识：代数系统定义在无限集上（正整数、自然数）。程序只能取有限子集（如前 N 个元素）进行验证。训练近似验证的边界认知：有限子集上结合律成立不代表全集合上成立；有限子集上未找到单位元也未必全集合上不存在（可能恰好漏掉）。这引入了测试与证明的差别，是计算思维区别于纯数学证明的关键。

3) 性质验证的模块化与层次化判定流程：将半群、独异点、群的判定拆解为独立函数：`is_associative()`, `has_identity()`, `has_inverse()`，再组合成高层分类。判定顺序可优化：先验结合律（若不成立直接非半群），再找单位元，再查逆元。这种分层短路避免了无谓计算，训练谓词逻辑的程序模块化，每个函数对应一条全称或存在命题的检查。

4) 结合律验证的算法设计与剪枝：朴素方法：三重循环枚举所有三元组 (a, b, c) ，检查 $op(op(a, b), c) == op(a, op(b, c))$ 。复杂度 $O(n^3)$ 。对于非结合的系统，一旦发现反例立即终止并报告，训练短路求值和早期失败策略。对于结合律成立的真半群，可引入优化：检查是否有快速结合性测试（如利用 Light's associativity test），训练超越蛮力的算法优化意识。

5) 单位元与逆元的存在性搜索：详见前例讲述的解决方法。

6) 反例的自动生成与诊断输出：当性质不满足时，程序不仅返回 `False`，还应输出具体反例。例如：“单位元不存在，因为对于候选 $e=0$ ，有 $op(0, 5)=6 \neq 5$ ”；“元素 1 无逆元”。这训练算法结果的可解释性，将内部搜索过程转化为用户可理解的证据，是构建可信系统的重要习惯。

7) 非交换性的判定：基于比较的搜索验证非阿贝尔群：检查是否存在 a, b 使得 $op(a, b) \neq op(b, a)$ 。二重循环遍历即可。若系统是阿贝尔群，则验证程序会穷举所有对且全相等，训练对“全称成立”的穷举验证能力。

8) 运算定义的语言映射与数据类型选择：正整数加法变种 $a + b + 1$ 直接映射为整数函数；自然数加法映射为内置 `+`；矩阵乘法需自定义二维数组乘法。选择合适的数据类型：整数、自定义矩阵类、甚至字符串（如前例）。训练将数学运算准确编码为无副作用、可复现的函数，避免浮点误差或类型错误。

9) 测试驱动的正确性自检：对于已知的经典代数系统（如模 4 加法群），先用程序验证确保判定逻辑正确，再应用于自构造的新系统。训练用已知真理校准工具的思维，确保验证器本身无误再用于探索未知。

总结：该案例通过“自构造代数系统→有限近似验证→性质分层判定→反例输出→性质对比”的完整闭环，将抽象代数中的群论公理映射为一套可执行、可诊断、可探索的算法框架，训练了数学建模、近似推理、搜索剪枝、模块化设计、可解释性验证等多维算法思维，是连

接数学抽象与计算实践的高级训练。

示例代码： 04.Function&Group/04.Constrat_Algebra.py

第五节 工程与算法综合一进阶训练

1、网络工程中的“高可用性(HA)拓扑”设计：命题逻辑在工程中的实践应用，是析取联结词 $\vee$ 、合取联结词 $\wedge$ 和复合命题结构的具象化应用。本质是网络结构双链路冗余（Active-Standby 或 Load Balance）设计，本质上是逻辑“或”/析取联结词的关系。串联的设备（如防火墙）本质上是逻辑“与”/合取联结词的关系。

提示：在网络设计中，我们经常使用冗余来消除单点故障（SPOF）。这在离散数学中可以直接映射为逻辑命题的合取（AND, $\wedge$ ）与析取（OR, $\vee$ ）。设计一个数据中心网络，有两条来自不同运营商的宽带（ISP A 和 ISP B）。内部有一台核心防火墙（FW）。目标是让服务器能上网。

算法思维与工程实践分析：

1) 数学建模：引导学生将现实世界转化为图论模型：并联（冗余）：只要 ISP A 或者 ISP B 有一个活着，外网就是通的。逻辑式： $(A \vee B)$ ； 串联（关键节点）：防火墙必须活着，流量才能通过。逻辑式： $(A \vee B) \wedge Fw$ 。

模型抽象：定义两种基本结构单元：Series（串联组）和 Parallel（并联组）。每个单元可以包含叶子组件（给定可靠性）或其他子结构，形成组合模式。

递归计算：

- (1) 叶节点：直接返回该组件的可用性值。
- (2) 串节点：计算所有子节点的可用性之积。
- (3) 并节点：计算 $1 - \prod(1 - \text{子节点可用性})$ 。

2) 工程实践的思维：如果防火墙出故障 ($Fw=False$)，不管 ISP 怎么冗余，网络全是断的。就是“与”逻辑造成的“单点故障”

- (1) 冗余设计权衡：并联冗余可指数级提高可用性（例如两个 99%的组件并联后可达 99.99%），但带来成本、复杂度增加。工具可快速对比不同方案的可用性。
- (2) 避免单点故障（SPOF）：任何串联路径上的单一组件都会成为可用性瓶颈。通过计算识别那些一旦移除就使整体可用性归零的组件，指导工程师增加冗余。
- (3) 分层建模：真实网络常按“接入→汇聚→核心”分层，每层内部可能有冗余，层间串联。将拓扑结构化，可逐层计算并定位薄弱环节。
- (4) 动态弹性设计：结合 SDN 或云环境，逻辑拓扑可动态调整（如故障时自动切换），工具可用于离线验证设计目标（如五个九的可用性是否可达）

示例代码： `Advanced_Case/11.Network_Logic.py`

代码运行的说明：程序的真值表计算，我们可以得出在 32 种可能的状态组合中，网络可用的场景只有 9 种（存活率 28.12%）。在网络工程中说明了一个重要问题：为了提高这 28.12% 的理论存活率，网络工程师必须在单点故障处（如上述拓扑中的接入交换机 SW）继续堆叠设备（例如使用 vPC/堆叠技术将单台 SW 变成两台物理机的逻辑组合），从而在离散命题中消除孤立的合取（AND）项，将其转化为析取（OR）项。

2、云服务器的“隐藏标价”——资源分配的对偶理论：假设云平台有 CPU、内存、GPU 三种资源，需要部署多种服务（如 Web 服务、AI 推理、数据分析），每种服务消耗不同资源并产生不同收益。在有限资源下，如何部署服务以最大化收益？（开发视角的产出最大化）；对偶问题 (Dual)：这些底层的计算资源到底“值多少钱”？最小化资源估值。（运维/CFO 视角的成本控制）；强对偶定理：最大化收益 $\equiv$ 资源的最小估值。对偶变量就是影子价格（破除瓶颈的指导价）。

提示：对偶思维，一个问题的两面：产出最大化 $\Leftrightarrow$ 成本最小化，殊途同归；影子价格 (Shadow Price) 约束边际价值，回答“多一份资源值多少钱”；互补松弛性：识别系统瓶颈——非零对偶变量 $\leftrightarrow$ 紧约束；强对偶定理：线性规划下 Primal 最优=Dual 最优，给出全局验证手段；敏感性分析：无需重解模型即可评估资源增减影响

算法思维与工程实践分析：

算法思维：

1) 对偶思维 (Duality Thinking) —— 换视角看问题，同一个问题，从“最大化产出”和“最小化成本”两个方向出发，会得到同样的最优值。培养学生的逆向思维解决问题的能力；

2) 影子价格思维 (Shadow Pricing) —— 稀缺性的量化，影子价格 = 松开一单位约束能带来的边际收益。它不是账面价格，而是机会成本。区分“标价”与“真实价值”，指导资源采购、定价、扩容决策。

3) 互补松弛性 (Complementary Slackness) —— 定位瓶颈，若某资源有剩余（松弛 >0 ），其影子价格必为 0；若影子价格 >0 ，该资源必被用尽。

4) 敏感性分析 (Sensitivity Analysis) —— what-if 推理，参数微扰后最优解如何变化？构建“稳健性”和“边际效应”的直觉，做决策前先算 ROI。

5) 抽象建模能力，把“服务部署”这种业务问题，抽象成矩阵 A，向量 b,c 的线性代数结构。从具体场景剥离出数学骨架，是该问题的核心目的之一。

6) 约束优化的思维范式，不再是“给算法求答案”，而是“给约束和目标，让求解器找答案”。这是声明式思维 (What) 对命令式思维 (How) 的胜利。

工程思维即决策分析：

1) 云资源定价：用影子价格给内部核算系统定“结算价”，让业务方感知真实成本

2) 容量规划：影子价格高的资源优先扩容（GPU 比 CPU 值钱 14 倍 $\rightarrow$ 该买 GPU）

3) 调度器设计：Kubernetes、YARN 的调度打分函数可借鉴对偶变量思想

4) 成本归因：FinOps 中把总账单按影子价格公平分摊到各业务线。

示例代码：详见 [Advanced_Case/12.Net_duality.py](#)

总结：“算力中心经常为了‘怎么把现有服务器榨干（原问题）’而绞尽脑汁。对偶问题 跳出了‘怎么部署’的细节，直接回答了一个更高维度的商业问题：系统里的每一种资源，到底暗含着多少经济价值（影子价格）？掌握了对偶，不仅知道程序该怎么跑，还知道明年的服务器预算该怎么花。”即对偶问题告诉你“资源值多少”，互补松弛告诉你“瓶颈在哪”，敏感性分析告诉你“下一步该往哪投钱”？

3、API 接口的“契约精神”与自动化测试，涉及的知识点有逻辑蕴含($P \rightarrow Q$)、真值表、前件/后件、空真 (Vacuous Truth)，结合领域是软件工程中的单元测试。

提示：在测试一个博客系统的后端 API。业务规则是如果用户是管理员 (P)，那么该用户拥有删除权限 (Q)。

算法思维与工程实践分析：

1) **数学建模**：其业务逻辑的表述形式是： $\text{IsAdmin}(\text{User}) \rightarrow \text{CanDelete}(\text{User})$

2) **算法思想与解决**。

Case 1 ($T \rightarrow T$)：是管理员，能删除。 -> 测试通过。

Case 2 ($T \rightarrow F$)：是管理员，系统提示“无权删除”。 -> BUG！测试失败！（这是我们要抓的错误）。

Case 3 ($F \rightarrow F$)：是普通用户，不能删除。 -> 测试通过（符合预期）。

Case 4 ($F \rightarrow T$)：是普通用户，竟然能删除？ ->

注意：从逻辑命题 $\text{IsAdmin} \rightarrow \text{CanDelete}$ 的角度看，这并没有违反规则（规则没说普通用户不能删，也许是他自己的文章）。

解决方案：用 Python 的 Assert 断言机制来实现蕴含逻辑的检查。逻辑等价变换：在代码中，很难写出 $\text{if } P \text{ implies } Q$ 。通常将 $P \rightarrow Q$ 转换为逻辑等价式：

$\neg P \vee Q$ ，这意味着要么不是管理员，要么有删除权限。也是程序中最常用的判断写法。

3) **蕴含的工程思维：**解释为什么 $F \rightarrow T$ 在逻辑上是 True。但在工程上，我们通常需要另一个规则（双向蕴含 $\leftrightarrow$ ）来约束普通用户。直观理解，把 $P \rightarrow Q$ 看作一种“承诺”或“契约”。在程序代码可视化结果中：

(1) 右上角的深绿色 User A，是管理员，也能删除。常规的测试通过。

(2) 右下角的红色区域 User B：是管理员 $P=T$ ，但不能删除 ($Q=F$)，这是唯一的 FAIL。因为这违反了系统承诺。这是 Bug 必须修复。

(3) 左侧区域浅绿色背景的 User C 和 User D，不是管理员 ($P=F$)，会发现不管 Q 是什么，结果都是 PASS。这就是离散数学中的“空真”(Vacuous Truth)。其工程意义：在编写测试脚本时，如果前提条件不满足（比如测试数据初始化失败，用户根本没创建成功），那么测试结果通常标记为 Skip 或 Pass，而不是 Fail，因为这并不能证明逻辑是错的。

只有当前提条件满足 P 却没有得到预期结果 Q 时，才判定系统有 Bug。其余情况，系统都是‘合规’的。”

示例代码：Advanced_Case/13.ImplicitApp.py

示例代码的运行说明：可视化图表直观地解释了蕴含式：

1) 右下角 (User_B) 红色区域。

状态：是管理员 ($P=T$)，但不能删除 ($Q=F$)。

分析：这是唯一的 FAIL。因为这违反了系统承诺。这是 Bug，必须修。

2) 左侧区域 (User_C, User_D) 浅绿色背景。

状态：是管理员 ($P=F$)。

分析：不管 Q 是什么，结果都是 PASS，这是离散数学中的“空真”(Vacuous Truth)。

工程意义：在编写测试脚本时，如果前提条件不满足（比如测试数据初始化失败，用户根本没创建成功），那么测试结果通常标记为 Skip 或 Pass，而不是 Fail，因为这并不能证明逻辑是错的。

3) 右上角 (User_A) 深绿色。

状态：是管理员，也能删除。

分析：常规的测试通过。

4、云计算集群的“看门狗”协议，涉及的核心知识点：一元谓词：P(x) (属性) 和 Q(x) (行为)；蕴含式：P(x)→Q(x)全称量词：∀x (对集群中所有服务器生效)；违反反例：∃x(P(x)∧¬Q(x))

提示：在像阿里云/AWS 这样的云平台中,为了保证网站不崩溃,系统监控着每一台服务器 x。设定的规则是对于所有服务器 x, 如果 x 的 CPU 负载过高 P(x), 那么必须触发自动扩容报警 Q(x)。

算法思维与工程实践分析：

1) 数学建模：论域 (Domain)：集群中的所有服务器集合 S={Server1,Server2,...Servern}。谓词定义：

P(x): CPU_Load(x)>80% (负载过高)
Q(x): Alert_Status(x)==True (报警已触发)。

其核心命题是：
∀x∈S,(P(x)→Q(x)), 即所有服务器必须遵守“高负载即报警”的协议。
采用真值表的形式予以分析，并给出最终的算法逻辑。

CPU 负载	报警状态	P(x) → Q(x)真值	解释
高 (T)	开 (T)	True	正常工作：负载高了，报警响了。
高 (T)	关 (F)	False	严重故障 (P0 级事故)：服务器快出故障，但监控没反应！这是逻辑上唯一的“假”。
低 (F)	关 (F)	True	正常静默：没负载，不报警，为了省钱/省事。
低 (F)	开 (T)	True	误报 (False Positive)：虽然很烦人（浪费运维精力），但并没有违反“高负载必须报警”这条安全规则。逻辑上它是“真”的（空真）。

2) 算法思想与解决，如何编写一个“合规性检查器”？不是检查哪个服务器正常，而是寻找反例 (Counter example)。根据谓词逻辑的否定律：¬(∀x(P(x) → Q(x))) ≡ ∃x(P(x) ∧ ¬Q(x))。算法逻辑是遍历所有节点，寻找满足 CPU>80∧Alert==False 的节点。

3) 蕴含的工程思维：“作为一名可靠性工程师 (SRE)，工作本质上就是在消除那个红色区域。一元谓词逻辑告诉我们：使用者不关心也没精力去验证每一个‘正常’的节点，我们只需要用 ∃x(P(x) ∧ ¬Q(x)) 去抓住那个让系统崩溃的‘反例’。设计 OR、AND 等逻辑表达式时，应注意哪些导致逻辑空真的条件。特别是程序设计语言中的惰性运算的逻辑空真。

示例代码：Advanced_Case/14.Unary_Predicate_Logic.py

示例代码的运行说明：在对应程序运行的示意图中分别有不同的表示：

- 1) 右下角 (红色区域 - Violation Zone): CPU 已经到了 90%甚至 100%，但是 Alert 状态都在 0 (OFF)。逻辑：T→F 为假。含义是这就是我们要找的 Bug（逻辑空真）。全称量词 ∀被证伪了。只要图中出现哪怕一个红点，整个系统的可靠性测试就是“不通过”。
- 2) 右上角 (绿色点): CPU 高，Alert 开。逻辑 T→T 为真。含义是系统按预期工作，成功扩容。
- 3) 左侧 (浅绿色点): CPU 低于 80%。此时无论 Alert 是开是关，点都是绿色的。逻辑是 F→T 或 F→F。含义是
 - (1)Alert=0 是正常的；

(2) Alert=1 (CPU 很低却报警) 是误报。虽然是工程瑕疵,但在当前这条逻辑规则下,它不算违规(因为规则没说“低负载不许报警”)。这能很好地帮助学生区分“逻辑真”与“业务最优”。

可视化图表将被分为三个明显的逻辑区域。

1) 右下角 (红色区域 - Violation Zone)

现象: CPU 已经飙到了 90% 甚至 100%, 但是 Alert 状态都在 0 (OFF)。

逻辑: $T \rightarrow F$ 假。

工程含义: 这就是要找的 Bug。全称量词 $\forall$ 被证伪了。只要图中出现哪怕一个红点,整个系统的可靠性测试就是“不通过”。

2) 右上角 (绿色点):

现象: CPU 高, Alert 开。

逻辑: $T \rightarrow T$ 为真。

工程含义: 系统按预期工作, 成功扩容。

3) 左侧 (浅绿色点):

现象: CPU 低于 80%。此时无论 Alert 是开是关, 点都是绿色的。

逻辑: $F \rightarrow T$ 或 $F \rightarrow F$ 。

工程含义:

(1) Alert=0 是正常的。

(2) Alert=1 (CPU 很低却报警) 是误报。虽然是工程瑕疵,但在当前这条逻辑规则下,不算违规(因为规则没说“低负载不许报警”)。这能很好地帮助学生区分“逻辑真”与“业务最优”。

5、数据库规范化——基于函数依赖的属性闭包与候选键。

在软件的数据建模阶段,设计关系数据库模式时,需要根据语义定义:在数据库系统中称为函数依赖 (Functional Dependency, FD), 例如: 学号 $\rightarrow$ 姓名。函数依赖是定义在属性集合上的二元关系, 满足自反、传递等推理规则 (数据库系统中的 Armstrong 公理系统)。避免数据冗余和更新异常, 需找出关系的候选键, 本质是计算**属性集闭包**, 寻找能决定全部属性的最小属性集。

提示: 一个属性集合 $U = \{A_1, A_2, \dots, A_n\}$ 。从业务语义中, 提取出形如 $X \rightarrow Y$ 的函数依赖 (Functional Dependency, FD), 其中 X 和 Y 都是 U 的子集。例如 $\{\text{学号}\} \rightarrow \{\text{姓名}, \text{系名}\}$ 。

两个核心的离散数学问题:

1) **属性闭包:** 给定一个属性子集 X , 在依赖集 F 的约束下, X 可以“逻辑决定”的最大属性集合 X^+ 。这正是关系传递闭包的一个具体体现——我们反复应用 F 中的规则进行推导, 直到无法推出新属性为止。

2) **候选键:** 能唯一标识元组的最小属性集。在数学上, 候选键 K 满足: $K^+ = U$, 且不存在 $K' \subset K$ 使得 $K'^+ = U$ 。这本质上是寻找一个最小集合, 使其闭包覆盖全集, 是典型的极小覆盖问题, 利用了集合包含关系进行剪枝。

算法思维与工程实践分析:

1) 算法思维: 首先是满足自反和可传递特性, 在此基础上还要求满足一个特殊得定律即增广特性: 若 $X \rightarrow Y$, 则有 $XZ \rightarrow YZ$ 。在算法中, 当我们发现 $\text{lhs} \subseteq \text{closure}$, 就自动把 rhs 加入 closure , 相当于用已有的 closure 去增广左部。

2) 不动点迭代：算法是一个经典的不动点计算。每次迭代，我们都根据当前 `closure` 运用所有函数依赖进行扩张。当某一轮没有任何新属性加入时，就达到了最小不动点，即 X^+ 。这确保了正确性，因为函数依赖推导出的所有属性都不会被遗漏。

3) 候选键求解：枚举、极小性检查与剪枝：

- (1) 遍历策略：按属性集大小从小到大枚举所有非空子集。这是基于极小性的必然选择：我们首先找最小的可能键，一旦找到，所有真超集都不可能是最小的，因此可以终止整个搜索；
- (2) 剪枝条件：利用了集合包含关系。假设已经找到了一个候选键 c (大小为 k)，此时正在检查一个大小为 $k' > k$ 的子集 `subset`。如果 $c \subset \text{subset}$ ，那么 `subset` 一定不是最小的，可以直接跳过，避免了昂贵的闭包计算。这虽然在某些极端情况下仍可能不完美（例如另一个同大小但不同的键不会剪掉超大集合，但因找到任意候选键就 `break` 了，所以实际剪枝效果极佳）。

总体上将逻辑推理(Armstrong 公理)转化为可执行的循环，利用集合包含关系设计剪枝，降低组合爆炸的影响。它体现了符号计算、不动点迭代、组合搜索与剪枝等核心思想。

4) 蕴含的工程思维：设计自动化与正确性保证：手工寻找候选键在大型模式（如上百个属性、几十个依赖）中几乎不可能不出错。将此代码嵌入数据库设计工具；可维护性与变更影响分析：当业务需求变更，增加新属性或新依赖时，只需重新运行求解器，即可即时看到候选键的变化，并评估对现有表结构和索引的影响。测试与边界：该模块可成为数据库 CICD 流水线的一环，自动对每次模式迁移脚本运行，检测是否存在非规范化设计或循环依赖（尽管函数依赖一般不循环）。

示例代码： [Advanced_Case/30.Database_Nomaliztion.py](#)

示例代码运行说明：

属性闭包算法严格遵循 Armstrong 公理，通过不动点迭代确保推导出所有可被 `attrs` 决定的属性，测试中所有闭包结果均符合预期。候选键查找采用按大小升序枚举 + 真超集剪枝的策略：

- 1) 测试 1、2、6 验证了单键和复合键的正确性。
- 2) 测试 3 验证了多个候选键（同层）均能被发现，且极小性得到保证。
- 3) 测试 4 在含依赖环且属性较多时，仍能找出全部极小键，并通过程序自动验证每个候选键的极小性（去掉任意属性后不再是超键）。
- 4) 测试 5 的极限情况（无函数依赖）正确返回整个属性集作为唯一候选键。
- 5) 剪枝有效性：测试 4 中候选键大小为 2，算法在第一层（`size=1`）未找到任何键后进入第二层，并通过子集剪枝避免了大量冗余计算；最终结果由断言验证了极小性。
- 6) 工程可用性：代码可直接嵌入数据库设计工具或课程教学演示，输入任意合法的属性集和依赖集，输出候选键，为后续范式检测提供基础。

6、模拟基于偏序的定时投入关联网络作业调度。在大型作业管理或操作系统中，存在大量互相关联的作业（如数据处理流水线、分布式计算任务）。这些作业具有明确的依赖关系（例如，作业 B 必须在作业 A 完成后才能启动），同时每个作业有各自的预定投入时间（提交时间）。若调度不当，会导致资源空闲等待、总完成时间延长。**模拟程序**要求把依赖关系建模为偏序集，利用阶位值（最长依赖链深度）指导调度，生成满足约束且高效的执行序列（耗时最短）。

提示：

1) 偏序建模：以每个作业为一个元素，依赖关系 $A \rightarrow B$ 表示 A 先于 B 执行，满足自反、反对称和传递性，构成偏序集关系；阶位值计算：在 DAG 中从任意源节点（无依赖作业）出发，沿依赖方向累加边数，到达某节点的最长路径长度即为该作业的阶位值（rank）。rank 越大表示依赖链越深。

2) 采用的调度策略，使用事件驱动的单 CPU 模拟，当前时间推进：

- (1) 当作业的所有依赖已完成且达到其提交时间时，进入就绪队列。
- (2) 就绪队列按 (rank, id) 升序排列（rank 小的优先执行，以减少后续作业的等待）。
- (3) 执行作业并更新完成集合，解锁新就绪作业，直至全部完成

算法思维与工程实践分析：

1) 作业的偏序建模：如何将实际问题映射为偏序结构，使得关系满足自反性、反对称性和传递性。依赖关系本质上是定义在模块集合上的二元关系。我们用邻接表存储关系图，用入度表示关系的“被依赖”次数。图的拓扑排序就等价于寻找该偏序关系的一个线性扩展。

2) 递归分治策略：高维偏序问题的降维处理，将“多维约束”分解为“一维排序 + 二维数据结构”的组合。采用 Kahn 算法（BFS），核心是每次移除入度为 0 的节点，并更新其邻点的入度。这相当于反复选择极小元，最终得到全序。

3) 并发与因果关系推理：判断偏序关系两个事件是否存在因果关系，如何理解计算机、网络工程里的并发事件。

4) 若排序结果包含的节点数少于总节点数，剩余的节点必然构成有向环，因为它们在去除所有无依赖节点后入度始终不为 0。这利用了集合运算（全集合与已排序集合的差集）。

5) 偏序与 DAG 的等价转化：有向无环图（DAG）是偏序关系的天然图表示。任何 DAG 都对应一个偏序集（传递闭包），任何有限偏序集都可用 DAG（哈斯图）表示。训练学生在 DAG 上执行拓扑排序、最长路径、关键路径等算法应用能力。

6) 工程思维：能够在偏序约束下的优化决策能力：在满足偏序约束的前提下，如何优化某个目标函数（如最小化总完成时间、最大化吞吐量）。

总结：偏序关系是连接离散数学理论与工程实践的一座桥梁。在软件工程中管理任务依赖、理解分布式系统中的因果关系、优化数据库查询；在网络工程中，它支撑着路由保护、通信调度等关键基础设施。掌握偏序相关的算法思维——从建模到拓扑排序、从分治到并发推理——不仅是对理论知识的深化，更是解决复杂工程问题的核心能力。

示例代码：详见 [Advanced_Case/31.Partial Order_set.py](#)

示例代码运行说明：代码模拟给出了偏序集模拟三组作业调度，依赖作业调度关系（非时间戳大小），给出三组最优的调度策略和提交作业到最小时间代价的时间长度。

7、在线协作文档、分布式数据库的多个节点可能同时修改同一份数据。传统做法依赖中央服务器或分布式锁，但会带来性能瓶颈和可用性问题。CRDT (Conflict-free Replicated Data Type, 无冲突复制数据类型) 是一种能在没有中央协调的情况下，让多个副本自动收敛到一致状态的数据结构。模拟实现在线文档协作处理的程序。

提示：其数学基础正是半群（特别是交换半群）即合并操作满足结合律（保证不同顺序合并结果相同）和交换律（保证并发修改不冲突）。以最简单的 CRDT——GCounter（增长计数器）为例：1) 每个节点维护一个自己的计数器（payload 是一个向量，每个分量代表一个节点的

计数值); 2) 增量操作只修改自己对应的分量; 3) 合并操作 (merge) 对每个分量取最大值; 4) 总值=所有分量的和。merge 操作是一个交换半群运算: 结合律保证多节点任意顺序合并结果一致; 交换律保证并发合并无冲突。

算法思维与工程实践分析:

1) 状态合并的数学基础: 所有副本的状态构成一个连接半群: 合并操作满足结合律、交换律、幂等律, 并存在一个最小状态(空文档)作为幺元。合并时求两个状态的最小上界(LUB), 保证了无论以何种顺序合并, 所有副本最终收敛到同一个不动点。

2) 副本间的更新事件构成偏序(happens-before 关系), 向量时钟是其标准实现: 判定两个操作是因果关系还是并发, 本质上是向量时钟的偏序比较。这决定了合并时采用哪种规则(例如保留两者或按规则消解)。

3) 交换律与结合律——无冲突设计的核心: CRDT 的精髓在于让并发操作可交换: $op1 \circ op2 = op2 \circ op1$ 。对于文档插入, 通常给每个字符分配全局唯一的全序标识(如(时间戳, 节点 ID) 或(序列号, 节点 ID)), 使并发的插入能以确定顺序排列, 而不会覆盖。删除通常采用墓碑标记(向集合添加“已删除”标记, 而非真正移除), 配合两阶段集合等代数结构, 保证删除与插入可交换。

4) 字符定位的确定论: 在文档协作的序列 CRDT 中(如 RGA、WOOT、Logoot), 每个字符携带一个唯一标识, 所有字符组成一个全序集。插入位置通过锚定邻居标识来实现, 即使并发插入也能通过标识的全序关系确定最终文档顺序。

5) 文档的内部结构表示: 文本 CRDT 使用树或有向无环图来组织字符顺序, 如 Logoot 使用区间树, RGA 使用链表/树。插入和删除操作转化为对图的局部修改, 最终文档由遍历图生成。

总结: 该案例将离散数学中的偏序、全序、格、图论与代数运算律等概念, 直接转化为分布式系统的一致性与协同算法设计。模拟实现过程训练学生从“数学性质推导”走向“利用性质构造系统”, 是训练计算思维与数学思维深度融合的典范

示例代码: 详见 [Advanced_Case/40.Muti_Docunment.py](#)

8、在大数据处理中, 经常需要对海量数据进行聚合统计(如求和、计数、最大值等)。MapReduce 等并行计算框架的核心思想是将计算拆分为可并行的子任务, 再合并结果。

提示: 含幺半群满足结合律, 保证了子任务可以任意划分、任意顺序合并; 单位元处理空数据集, 不影响最终的运算结果。

算法思维与工程实践分析:

1) 代数结构与可并行性——交换半群的应用: MapReduce 对聚合操作(求和、计数、最大值、集合并等)的并行化依赖于交换律与结合律。本质上是把数据视为一个交换半群(有结合律与交换律, 不一定有单位元)上的元素, 归约过程等价于在半群中计算所有元素的“乘积”

2) 单位元与可组合性——幺半群的作用: 实际实现中往往还需要单位元(求和为 0、求积为 1、最大值为 $-\infty$ 、集合并为空集), 使得空分区也能返回有意义结果, 便于组合。这对应于交换幺半群(Commutative Monoid)。MapReduce 是自动将函数在数据上折叠为幺半群的累加结果。

3) 分治法与归约树——结合律的计算: 结合律允许计算被组织为任意树形结构的归约,

而交换律允许数据任意分布。这使得计算可拆分为 map 端的局部归约 (Combiner) 与 reduce 端的全局归约。

4) Map 阶段: 函数映射与笛卡尔积分解: Map 将原始数据映射为键值对, 本质上是集合上的函数或关系的分解。当需要多表关联时, 则涉及笛卡尔积与连接操作。将数据处理过程形式化为映射、过滤、投影等离散数学操作, 利用关系代数的组合规则来设计 Map 阶段。

5) 复杂度模型与通信-计算权衡: MapReduce 涉及排序、磁盘 I/O、网络 Shuffle, 复杂度分析不仅是计算量, 更关注通信复杂度与轮数。通过 Combiner 局部归约减少传输量, 就是利用结合律优化通信。

总结: MapReduce 不仅是一个工程框架, 更是一个基于交换幺半群、等价划分、图论依赖、哈希映射等离散数学原理构建的抽象计算模型。深入理解这些数学根基, 才能真正掌握分布式算法的设计与分析方法, 形成从抽象代数到系统架构贯通的计算思维。

示例代码: [Advanced_Case/41.Sumulite_Mapreduce.py](#)

9、半群理论的应用案例—Git 的版本合并与“半群”特性, 也是偏序关系的工程级应用, 在 Git 协作开发中, 多个特性分支并行开发, 提交历史形成有向无环图 (DAG)。两个分支上的提交 (如 C1 和 C2) 在合并前是“不可比”的 (偏序关系)。这种网状结构在代码审查、回滚和 CI/CD 触发时会造成混乱。git rebase 的核心作用正是打破这种不可比性, 将偏序历史强制转化为严格的全序线性历史。

提示: Git 的版本合并基于文本差异, 将提交视为对象, 合并操作作为二元运算。要求封闭性和结合律, 合并两个提交产生一个新的合并提交, 这个新提交属于提交集合, 因此封闭。在某些条件下 (如无冲突且合并顺序无关紧要) 可能满足结合律。

算法思维与工程实践分析:

1) 偏序与格——快进合并的代数基础: 提交之间的父子关系构成偏序 (happens-before 关系), 整个历史是一个 DAG。当两个分支不存在分叉 (即一个分支是另一个的祖先) 时, 合并运算表现为取最小上界 (即较新的提交), 赋予提交集合一个半格结构。

2) 合并运算的封闭性与幺半群证明: 合并两个提交产生一个新提交, 该提交仍属于提交集合, 满足封闭性。在某些受限条件下 (无冲突且忽略合并顺序差异), 合并可视为满足结合律的运算, 从而构成半群或交换半群。当把“不合并”视为单位元时, 甚至可尝试构造幺半群。

3) 图论中的最近公共祖先: 合并的核心是找到两个提交的最近公共祖先 (LCA), 并使用三路合并算法。

LCA 问题是有向无环图中的可达性与汇聚点查询, 可以用 Tarjan 算法或倍增法求解。将版本历史映射为 DAG, 并运用图论算法解决“共同基准”的计算, 理解 DAG 中 LCA 与树中 LCA 的差异。

4) 冲突检测与合并的交换律缺失: 当两个分支对同一行进行不同修改时, 合并产生冲突, 且合并顺序不同可能导致解决结果不同, 破坏了交换律。无冲突的合并通常具有交换性, 这可通过三路合并的对称性证明。通过分析合并运算是否满足交换律, 来界定冲突的本质, 并将冲突视为代数运算的非交换性表现, 深化对运算性质的理解。

5) 内容寻址与不可变性——哈希函数的确定性: Git 用 SHA-1 哈希值标识提交, 哈希函数是确定性的单射映射 (从内容到哈希值), 这保证了提交的不可变性和唯一性。将哈希视

为函数映射，理解内容寻址如何利用数学上的确定性构建去中心化的对象存储，以及哈希碰撞对系统的影响。

6) 拓扑排序与可达性——Git 的图算法核心：git log 的排序、分支追踪、可达性判断等操作依赖于 DAG 的拓扑排序与可达性分析。合并提交的引入产生了多个入度，使得遍历算法需要考虑多父节点。将图论中的遍历、拓扑排序、强连通分量等概念直接应用于版本历史的查询与分析。

7) 递归合并与分治法——合并策略的算法设计：Git 的递归合并策略在面对多次交叉合并时，会递归地合并共同祖先以产生虚拟祖先，这本质上是分治法在合并场景的应用。将合并问题分解为子合并问题的递归求解，理解“合并历史”本身也是可合并的对象，体现了算法分层与递归思想。

总结：该例子中的 Git 版本控制系统将偏序与格、图论、哈希、递归与拓扑排序等离散数学概念整合在一个实用工具中。通过将提交视为对象、合并视为二元运算，并分析其代数性质，学生能深刻体会到抽象代数与图论如何支撑现代软件工程的协作基础，训练从数学结构推导系统行为的核心计算思维。

示例代码： 详见 [Advanced_Case/42.Semi_Group_Git.py](#)

10、集合中任意三数之和为一个固定值（如经典的零和问题）：将“加法”推广为群论中的二元运算，将集合推广为代数结构。“零和定理”定理是指在任意的 $2n - 1$ 个整数集合中，必然可以找到恰好 3 个数，它们的和是 0。

提示：给定一个集合（数组）和一个目标值 $K=0$ ，找出数组中所有和为 K 的不重复的三元组。暴力破解（三重循环）的时间复杂度是 $O(N^3)$ 。最优的实用算法是“排序 + 双指针 (Sorting + Two Pointers)”，可以将时间复杂度降到 $O(N^2)$ 。

算法思维与工程实践分析：

1) 从具体运算到抽象运算的泛化能力：经典问题中加法具有交换律、结合律，群运算可能非交换、无全序。

必须将“和为 0”转化为“乘积为 e ”，并基于群公理（结合律、单位元、逆元）来设计算法，而非依赖算术加法性质。训练学生能将具体问题抽象为代数结构上的通用算法，学会利用接口（如乘法函数、相等性判断）代替硬编码运算。

2) 问题等价转化与约简——结合律与逆元的运用：在群中 $a \cdot b \cdot c = e \Leftrightarrow a \cdot b = c^{-1}$ 。若群有交换律（阿贝尔群），还满足 $c = (a \cdot b)^{-1}$ 可直接哈希查找。训练学生利用运算性质进行问题重述，将三层循环减为两层循环加查表，是“空间换时间”的典型映射。

3) 哈希查找与通用标识——元素去重与索引管理：需要快速查找某个元素是否存在以及所在位置。群元素不具备天然的全序，不能用排序或双指针，哈希表（字典）成为主要工具。

4) 暴力穷举的剪枝与搜索空间压缩：朴素 $O(n^3)$ 搜索可被大幅剪枝：固定 a, b 时，只需计算目标 $c = (a \cdot b)^{-1}$ ，直接在哈希表中检查，复杂度降至 $O(n^2)$ 。还可进一步优化：若群很小（有限群），可预计算全体元素的逆元表；若群具有正则表示（如置换群），可提前编码。利用目标导向的约束生成减少搜索维度，以及“先计算条件，再验证”的反向思维。

5) 非交换群带来的顺序敏感性与双向检验：对于非阿贝尔群， $a \cdot b \cdot c = e$ 的顺序固定。若考虑所有排列（如

$a \cdot c \cdot b = e$ 等），则需分别检验不同顺序。或转化为检查 $a \cdot b = c^{-1}$ ，此式在非交换下唯一确定。寻找三元组时，必须严格按群运算顺序处理。

6) 利用生成元与正规形加速等值判断：若群元素可用符号表示（如自由群的简化词、对称群的轮换分解），可将元素转换为标准形式（正规形），加速相等性检查与哈希映射。对于某些无限群（如长度 ≤ 5 的字符串），运算可能产生增长，此时需结合抽样搜索。规范化表示在算法中的应用，通过将元素转化为唯一规范形式来简化比较和存储。

7. 从算法到存在性验证的思维跨越：如果问题背景是验证“零和定理”（如给定 $2n-1$ 个元素必有三元组乘积为 e ），程序可扮演反例搜索的角色：随机生成大量测试集，检查是否总能找到满足条件的三元组。若发现反例，可输出；若长时间未发现，可增强对定理的置信度。训练实验数学与计算证明的思路：当完全证明困难时，用程序进行统计验证与边界探索。

8) 复杂度分析与群运算代价的权衡：群乘法本身可能复杂度较高（如矩阵乘法、字符串拼接），因此 $O(n^2)$ 次群运算可能是实际瓶颈。或利用分组聚合方式批量处理降低为 $O(n)$ 。训练将运算代价计入复杂度分析，在时间与空间、预处理与查询之间做出权衡。

总结：该题看作是一个在群论中满足消去律的案例，只有满足群的定义可以使用消去律。将经典三数之和问题从整数加法迁移到抽象群，带来了运算非交换、元素无序、哈希依赖等新挑战，全面训练了代数建模、问题等价转化、哈希搜索、群性质利用和实验数学验证等核心算法思维，是从具体数据结构走向抽象代数计算的典型桥梁。

示例代码：详见 `Advanced_Case/43.ThreeData_Add_equal_zero.py`

11、研讨会上的座位问题：现实中的“共同语言”被抽象成了图论中的“边”。问题本质上是在问“这个是否构成一个可以相互连接的图？共同语言连接的图是否是一个哈密尔顿图？”

提示：A, B, C, D, E, F, G 七个人参加一个国际研讨会，其中 A 会讲英语，B 会讲英语和汉语，C 会讲英语、意大利语和俄语，D 会讲日语和汉语，E 会讲德语和意大利语，F 会讲法语、日语和俄语，G 会讲法语和德语。能否安排这七个人围绕圆桌而坐使每个人都能和他两旁的人交谈。约束是要求安排这七个人围绕圆桌而坐使每个人都能和他两旁的人交谈。从某人开始，经过每个人一次且仅一次，最后回到起点的路径，就构成了圆桌座次。

算法思维与工程实践分析：

1) **数学建模**，引导学生将现实世界转化为图模型结构：

- (1) 顶点 (Vertices/Nodes): 参加会议的 7 个人 (A, B, C, D, E, F, G)。
- (2) 边 (Edges): 相邻两位在语言上有交集（能交流），则在他们之间连一条边。

2) **算法思想与解决**。深度优先搜索 (DFS) + 回溯法 (Backtracking)。通俗来说，就是“一条路走到黑，撞了南墙再回头”。

● **试探（深度优先搜索 DFS）**，计算机开始尝试排座位，假设从 A 开始：

- (1) A 坐下。
- (2) 查找 A 的邻居（能交流的人），假设选了 B。
- (3) 查找 B 的邻居（且没坐下的），假设选了 D。
- (4) 查找 D 的邻居...
- (5) 这样一直顺着链条往下找。

● **回溯（后悔机制）**是算法最关键的部分。假设排到了第 6 个人 F，轮到给 F 找邻居时发现：

- ✓ 情况一：F 周围能交流的人都已经坐下了（没有空位了）。
- ✓ 情况二：虽然找到了第 7 个人 G，但 G 和第一个人 A 没法交流（无法闭合圆圈）。

此时算法判定“此路不通”。它会退回到上一个人（比如 E），让 E 不要选 F，而是换一个邻居试试。如果 E 换了所有邻居都不行，就再退回到 D…。直到找到一条路径，使得 7 个人全部坐下，且首尾相连。

3) 工程理论分析：这是计算机算法思维中最典型的一个应用，深度优先搜索（类似的还要广度优先搜索）+递归和回溯算法，是工程项目开发中最最基础的应用之一。从一个复杂全连接网状拓扑（Mesh Topology），简化为一个哈密尔顿图。如果是人脑解题，通常会用度数约束走捷径：

- (1) 看 A，他只会英语。
- (2) 全场只有 B 和 C 会英语。
- (3) 所以 A 的两边必须是 B 和 C（A 只有两条路）。
- (4) 确定了 B-A-C 的结构，剩下的只是怎么把剩下 4 个人填进缺口里。比计算机盲目搜索要快得多。

示例代码： [Advanced_Case/21.Create_Tree\(Hamiltonian Cycle\).py](#)

运行说明：

1、台输出——成功找到圆桌座位安排方案！

A --(English)--> B --(Chinese)--> D --(Japanese)--> F --(French)--> G --(German)--> E --(Italian)--> C --(English)--> A

注：由于算法搜索顺序不同，可能会得到该序列的逆序或以不同人开始的序列，但邻居关系是不变的

2、视化图表——看到 7 个圆点排成一个完美的圆圈（代表圆桌）。

红色实线连接了所有人，形成一个闭环。连线上的文字**清晰地标明了这两人是用什么语言交流的：

- A 和 B 聊英语
- B 和 D 聊汉语
- D 和 F 聊日语
- F 和 G 聊法语
- G 和 E 聊德语
- E 和 C 聊意大利语
- C 和 A 聊英语

12、教学案例名称：校园光纤网络铺设方案：涉及的核心知识点：加权无向图、连通性、最小生成树 (Kruskal 算法/Prim 算法思想)

提示：现实问题：假设你是一所新建学校的网络工程师。校园里有 5 栋主要建筑（行政楼、图书馆、教学楼、宿舍、食堂）。工程的目标是需要铺设光纤将这 5 栋楼全部连通，使得任何两栋楼之间都可以进行数据传输（可以直接连，也可以通过中间建筑转发）。成本约束是因为铺设光纤非常昂贵，成本取决于距离和施工难度。如何在保证所有建筑连通的前提下，使得总施工成本最低？

算法思维与工程实践分析：

1) 数学建模，引导学生将现实世界转化为图论模型：

- (1) 顶点 (Vertices/Nodes): 5 栋建筑。
- (2) 边 (Edges): 建筑之间可能铺设的路径。
- (3) 权重 (Weights): 每一条路径的铺设成本（单位：万元）。
- (4) 图的类型：这是一个加权无向图 (Weighted Undirected Graph)。

(5) 概念映射:

(6) 连通图: 任意两点可达 (网络通了)。

(7) 生成树 (Spanning Tree): 包含图中所有顶点, 且没有回路 (环) 的子图。为什么不能有回路? 因为回路意味着冗余的线路, 增加了不必要的成本。

(8) 最小生成树 (MST): 在所有生成树中, 边权之和最小的那一棵。

2) 算法思想与解决, 采用贪心策略:

(1) 既然要省钱, 就先挑最便宜的路修。

(2) 如果这条路连接了两个还没有连通的区域, 就选它。

(3) 如果这条路加上会形成一个圈 (回路), 就不要选 (因为浪费)。

(4) 重复直到所有楼都连上。

3) 工程理论分析: 详见前题图论中 Kruskal 算法核心思想。拓扑结构变化: 从一个复杂的全连接网状拓扑 (Mesh Topology), 简化为一个树状/星型混合拓扑, 这是网络物理层设计的基础。

示例代码: [Advanced_Case/22.MST_application.py](#)

运行说明:

● 左图 (原始图)展示了所有可能的连接方式。这就是一个高度冗余的 Mesh 拓扑。虽然可靠性极高 (一条路断了有备用), 但全部铺设成本太高 (计算一下所有边权重之和是 54 万)。

● 右图 (最优拓扑)红色的实线展示了计算出的 MST。没有任何闭合的回路 (Loop), 这就是“树”的特征。虽然 'Admin' 和 'Library' 之间有直连路径 (成本 4), 但算法选择了 'Admin' -> 'Classroom' -> 'Library' (2+3=5) 吗? 这里要注意, Kruskal 会选最短的边。

结果选了 Admin-Classroom(2), Library-Classroom(3), Dorm-Canteen(5), Classroom-Canteen(6) 。'Library' 到 'Dorm' 直连要 8 万, 太贵。方案是通过 Library->Classroom->Canteen->Dorm (3+6+5=14) 吗? 不, 是数据传输走这条路, 但物理连接只需要保证连通即可。最终总成本由 54 万降到 16 万, 这就是图论在工程预算中的威力。

13、无线基站的频谱 (频道) 分配, 涉及的核心知识点: 图的顶点着色 (Vertex Coloring)、色数 (Chromatic Number)、干扰图 (Interference Graph)

提示: 为一个科技园区部署 8 个 5G 基站 (或者 WiFi 热点)。但是存在一个物理限制, 即无线电频谱是有限的资源。如果两个距离很近的基站使用相同的频率 (频道), 它们会发生信号干扰, 导致用户网速变慢甚至掉线。设计的工程目标是给每个基站分配一个频道, 要求相邻 (互相干扰范围内) 的基站必须使用不同的频道。其次是考虑成本约束, 因为购买频段许可证非常昂贵。成本目标是使用尽可能少的频道数量来完成分配。

算法思维与工程实践分析:

1) 算法思维: 将物理布局转化为“干扰图”:

(1) 顶点 (Vertices): 每一个基站就是一个顶点。

(2) 边 (Edges): 如果两个基站距离过近 (存在干扰可能), 就在它们之间连一条边。

(3) 颜色 (Colors): 代表不同的通信频道 (频率)。

问题转化为经典的图着色问题。要寻找图的色数 (Chromatic Number, $\chi(G)$), 即染好这个图最少需要几种颜色。其拓扑意义是拓扑不是指物理连接线, 而是“冲突/约束拓扑”。

2) 工程思维: 算法上采用贪心策略 (Greedy Coloring), 虽然求绝对最小色数是 NP-Hard 问题, 但在工程上通常使用高效的贪心算法, 即将所有基站按“邻居数量”(度, Degree) 从多到少排序。(邻居多的基站是“刺头”, 最难搞, 先处理它)。依次为每个基站分配颜色。分配时, 查看它的邻居都用了什么颜色, 然后从颜色列表(频道 1, 频道 2, 频道 3...) 中选一个最小编号且未被邻居占用的颜色。

示例代码: [Advanced_Case/23.Interference Graph.py](#)

示例代码运行说明:

1) 左图 (干扰图) 这些点是基站的实际位置。红色的虚线**代表“冲突”。比如基站 0 和基站 1 连线了, 意味着如果我们给它俩分配同一个频道, 用户就会遇到严重的信号干扰。拓扑视角是一个高度互连的图, 如果要手动分配, 很容易顾此失彼。

2) 右图 (解决方案) 算法自动计算出了颜色分配方案。看任何一条连线的两端, 颜色一定是不一样的 (例如 0 号是红色, 1 号是紫色)。在这个案例中, 算法使用了 3 种颜色 (频道) 就完美解决了所有冲突。工程价值在于如果不使用图论算法, 可能为了保险起见, 给 8 个基站买 8 个频道, 成本极高。而现在只需要购买 3 个频段的许可证, 节省了 62.5% 的频谱成本。

14、数据中心巡检机器人的“双重任务”, 涉及的核心知识点: 欧拉回路 (Eulerian Circuit): 遍历所有边一次且仅一次 (关注的是“线”)。哈密尔顿回路 (Hamiltonian Cycle): 遍历所有顶点一次且仅一次 (关注的是“点”)。网络拓扑应用: 全链路压力测试 vs 令牌环/数据采集路由。

提示: 场景引入, 大型数据中心里面有服务器 (节点) 和连接它们的光纤网线 (边)。有一台智能巡检机器人。

1) 任务 A (线路质检模式): 为了防止老鼠咬坏线缆, 或者检测光纤衰减, 机器人必须走过每一条网线进行扫描, 最后回到充电桩。**问题: 能不能不走回头路 (重复路), 把每条线都查一遍?**

2) 任务 B (数据采集模式): 机器人需要走到每一台服务器前, 无线连接并下载日志数据, 最后回到充电桩。

问题: 怎么走路径最短? 即每个服务器只去一次, 不重复访问。

算法思维与工程实践分析:

算法思维:

1) 概念解析与拓扑结合:

第一部分: 任务 A 与欧拉图 (Eulerian),

定义: 经过图中每条边一次且仅一次的回路。

工程判据 (柯尼斯堡七桥问题的推广):

如果一个网络拓扑中, 每一个节点的度数 (连接的线缆数) 都是偶数, 那么这个网络一定存在欧拉回路。

拓扑启示: 如果你在设计一个需要经常全链路检测的网络, 最好保证每个节点的接口数是偶数, 这样检测效率最高 (无重复路径)。

第二部分: 任务 B 与哈密尔顿图 (Hamiltonian)

定义: 经过图中每个顶点一次且仅一次的回路。

工程判据: 不同于欧拉图, 哈密尔顿图没有简单的充分必要条件 (这是一个 NP-Complete 难题)。

拓扑启示：这是经典的旅行商问题 (TSP) 的特例。在网络设计中，这对应着令牌环网 (Token Ring) 的逻辑——令牌如何以最高效的方式传递给每一个站点然后回到原点。

对比：“欧拉图是邮递员，他必须走遍每条街道投信；哈密尔顿图是推销员，他只需要敲开每家客户的门，不在乎走哪条路。”

2) 工程实践的思想：Hierholzer 算法 (用于欧拉回路)：寻找圈，然后合并圈。回溯法/遗传算法 (用于哈密尔顿回路)：因为难解，工程上常用近似算法寻找最优解。

示例代码：Advanced_Case/24.EULA_application.py

示例代码运行说明：

1) 左图 (任务 A - 欧拉回路)会看到箭头颜色从青色渐变到紫色 (cool colormap)。机器人从起点出发。观察边：图上每一条黑色的连线都被彩色的箭头覆盖了。这意味着所有的网线都检测到了。观察点：有些服务器 (节点) 被经过了多次 (比如中间的节点)。这没关系，欧拉图允许重复经过点，只要不重复走线。原理是因为每个服务器连接的网线数量都是 4 根 (偶数)，进一次出一此，进两次出两次，刚好抵消。

2) 右图 (任务 B - 哈密尔顿回路)只有一圈红色的实线，背景有很多虚线 (未走的边)。机器人的目标变了。它只关心“人” (服务器)。红色的圈把 6 个服务器串成了一串项链。效率指它不需要走完所有的网线，那太浪费电了。它只要走通这一个圈，就能拿到所有数据。这就是计算机网络令牌环网 (Token Ring)的工作原理。